

2026 第二届大学生人工智能安全竞赛

作品报告

作品名称: AI-WAF: 针对混淆逃逸的 Web 攻击载荷动态检测与对抗系统

作品类型: 定向式专项命题

电子邮箱: x_yzy@qq.com

提交日期: 2026.8.2

填写说明

1. 所有参赛项目必须为一个基本完整的设计。作品报告书旨在能够清晰准确地阐述（或图示）该参赛队的参赛项目（或方案）。
2. 作品报告采用 A4 纸撰写。除标题外，所有内容必需为宋体、小四号字、1.5 倍行距。
3. 作品报告中各项目说明文字部分仅供参考，作品报告书撰写完毕后，请删除所有说明文字。（本页不删除）
4. 作品报告模板里已经列的内容仅供参考，作者可以在此基础上增加内容或对文档结构进行微调。
5. 为保证网评的公平、公正，作品报告中应避免出现作者所在学校、院系和指导教师等泄露身份的信息。

目录

摘要.....	1
第一章 作品概述.....	3
1.1 背景分析	3
1.1.1 Web 攻击载荷及其安全威胁.....	3
1.1.2 混淆逃逸的主要表现	4
1.2 相关工作分析	5
1.2.1 基于特征规则的检测方法.....	5
1.2.2 基于机器学习增强的检测方法	5
1.2.3 基于深度解码与语义分析的检测方法.....	6
1.3 作品描述	7
第二章 作品设计与实现	9
2.1 系统架构设计	9
2.1.1 设计目标	9
2.1.2 总体架构	9
2.1.3 系统工作流程.....	10
2.2 系统实现方案与功能	12
2.2.1 数据集与对抗样本构建.....	12
2.2.2 HTTP 请求解析与递归归一化	16
2.2.3 特征提取与双模型分析.....	19
2.2.4 多源融合与请求级判定.....	21
2.2.5 API 服务与前置代理防护	23
第三章 作品测试与分析	26
3.1 测试方案	26
3.2 载荷检测与对抗能力测试	27
3.3 组件消融与性能对比	28
3.4 与已有 WAF 产品的对比测试.....	31
3.5 HTTP 请求级动态检测测试	33
3.6 在线防护与运行性能测试	34
第四章 创新性说明.....	40
4.1 归一化与多视图检测	40
4.2 规则与双模型协同.....	40
4.3 多源混淆数据构建.....	41
4.4 请求级防护闭环.....	41
第五章 总结.....	42
参考文献	43

摘要

Web 攻击载荷可隐藏在 URL、请求路径、请求体、请求头和 Cookie 等位置，并利用多层编码、注释插入、大小写扰动、字符串拆分和路径变形改变外观。仅依靠关键词或固定规则，难以同时处理混淆变体、正常技术文本和完整 HTTP 上下文，容易产生漏报或误报。面向真实业务的防护系统需要还原隐藏语义、定位风险字段，并形成请求级结论。

本文设计并实现 AI-WAF，通过递归归一化、规则检测与机器学习融合识别混淆载荷，并对完整 HTTP 请求进行检测与拦截。系统解析请求并保留载荷原文及字段来源，以有边界的多轮归一化还原编码和结构变形；高置信规则先处理证据明确的输入，基于 38 维结构特征的 LightGBM 与字符 n-gram 模型再分析边界样本，最终结合字段位置与多载荷结果形成请求级判定。作品同时建立多来源数据整理、可复现混淆生成、双模型训练和独立评测流程。

在 34,721 条正式测试记录上，完整检测链路的精确率、召回率和 F1 分别为 99.0476%、98.8515% 和 98.9495%，误报率为 0.0745%；对 1,357 条混淆攻击的召回率为 98.8946%，载荷级 P99 延迟为 1.8383 ms。在统一数据和判定口径下，其 F1 高于 ModSecurity + OWASP CRS、open-appsec 和 SafeLine 三个固定版本产品。在含 89.4737% 正常流量的 9,500 条组件测试中，完整链路以 99.0010% 的 F1 将 P95 和 P99 控制在 0.6537 ms 和 1.2704 ms。

系统提供单载荷、批量载荷和完整 HTTP 请求检测 API，并可作为前置代理接入 Web 服务。在服务器测试中，XSS 请求在到达业务程序前被识别、阻断并记录，正常请求则被转发至上游，验证了从载荷检测到在线防护的有效性。

关键字： Web 应用防火墙 混淆逃逸 载荷归一化 对抗样本 机器学习

Abstract

Web attack payloads may appear in URLs, paths, bodies, headers, or cookies, while nested encodings, inserted comments, case changes, and string splitting can hide their structure. Keyword and fixed-rule methods may therefore miss obfuscated variants or misclassify benign technical text. A practical Web application firewall must recover hidden semantics, locate risky fields, and make request-level decisions.

This report presents AI-WAF, an AI-enhanced Web application firewall for obfuscation evasion. It parses complete HTTP requests, preserves raw content and field sources, and applies bounded recursive normalization. High-confidence rules handle clear cases first; uncertain inputs are analyzed by a LightGBM model over 38 structural features and a character n-gram model. Their evidence is combined with field context and multi-payload aggregation to form a request-level decision. The project also includes multi-source data preparation, reproducible obfuscation generation, dual-model training, and independent evaluation.

On 34,721 formal test records, the pipeline achieved 99.0476% precision, 98.8515% recall, 98.9495% F1, and a 0.0745% false-positive rate. Recall was 98.8946% on 1,357 obfuscated attacks, and payload-level P99 latency was 1.8383 ms. Under the same data and criteria, its F1 score exceeded those of fixed versions of ModSecurity with OWASP CRS, open-appsec, and SafeLine. In a separate 9,500-record component test containing 89.4737% benign traffic, the complete cascade retained 99.0010% F1 with P95 and P99 latencies of 0.6537 ms and 1.2704 ms.

AI-WAF provides payload, batch, and complete-request APIs and can operate as a reverse proxy. In a server test, an XSS request was detected, blocked before reaching the application, and recorded, while benign requests were forwarded normally. This verifies the effectiveness of the path from payload detection to online protection.

Keywords: Web application firewall; malicious payload detection; obfuscation evasion; normalization; adversarial samples; machine learning

第一章 作品概述

1.1 背景分析

登录、搜索、评论、文件上传等 Web 功能均通过 HTTP 请求接收外部输入。请求中的 URL、参数、请求体、请求头和 Cookie 可能被数据库、浏览器、文件系统或操作系统进一步解释；一旦应用未正确隔离“数据”与“指令”，输入便可能改变程序原有语义。部署在业务入口的 Web 应用防火墙（Web Application Firewall，WAF）可在请求到达应用前进行检测、告警或阻断，是业务安全开发与漏洞修复之外的重要纵深防线 [1]。

图 1.1 概括了 HTTP 请求进入业务系统的主要风险位置及本作品的检测范围。系统在 Web 服务入口统一检查路径、参数、请求体、请求头和 Cookie，并在危险内容到达后续解释环境前形成判断。

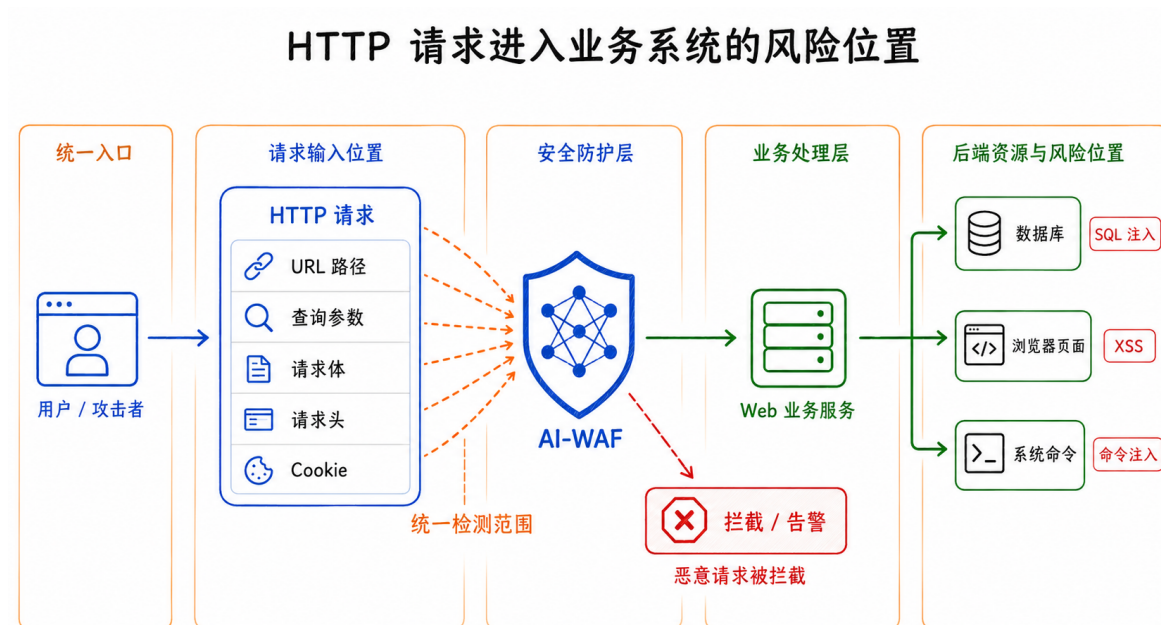


图 1.1 HTTP 请求进入业务系统的风险位置与 AI-WAF 检测范围

1.1.1 Web 攻击载荷及其安全威胁

攻击载荷（Payload）是攻击者为改变程序正常处理过程而构造的输入。SQL 注入利用引号、运算符或注释改变数据库查询语义，XSS 使用户输入被浏览器作为页面

代码解释，命令注入则借助连接符等结构追加系统指令 [4, 5, 6]。三类攻击作用环境不同，却都需经 HTTP 请求进入应用，因此请求入口提供了统一检测位置。

载荷中的引号、标签或关键词脱离上下文并不必然有害；技术文章、搜索词和合法业务参数也可能包含相同字符。故本作品判断的是请求是否表现出攻击意图，而非宣称攻击已经成功，并保留字段位置与请求结构作为判定依据。载荷检测不能替代参数化查询、上下文编码、最小权限等安全开发措施。

1.1.2 混淆逃逸的主要表现

混淆逃逸是指在基本保持攻击语义的前提下，通过大小写扰动、注释插入、字符拆分、URL/HTML 编码或语法等价改写改变载荷外观，以降低被检测概率。多种变换还能按不同顺序叠加，使固定字符串不再直接出现；WAF-A-MoLE、AdvSQLi 等研究已证明此类变换可绕过部分规则和学习型检测器 [17, 18]。

混淆利用的并非“加密”，而是组件间的解释差异。例如 %2527 经两次解码后恢复为单引号；若 WAF 只检查原文或仅解码一次，而后端继续解码，双方会对同一输入形成不同理解。字段位置、内容类型、同名参数处理和请求体解析差异也可能扩大这一偏差 [3, 19]。同时，URL 编码本身是正常传输机制，不能被直接视为攻击 [9]。检测系统因而既要恢复隐藏结构，也要保留原始视图和业务上下文，避免过度还原造成误报。

混淆攻击载荷的多视图还原过程

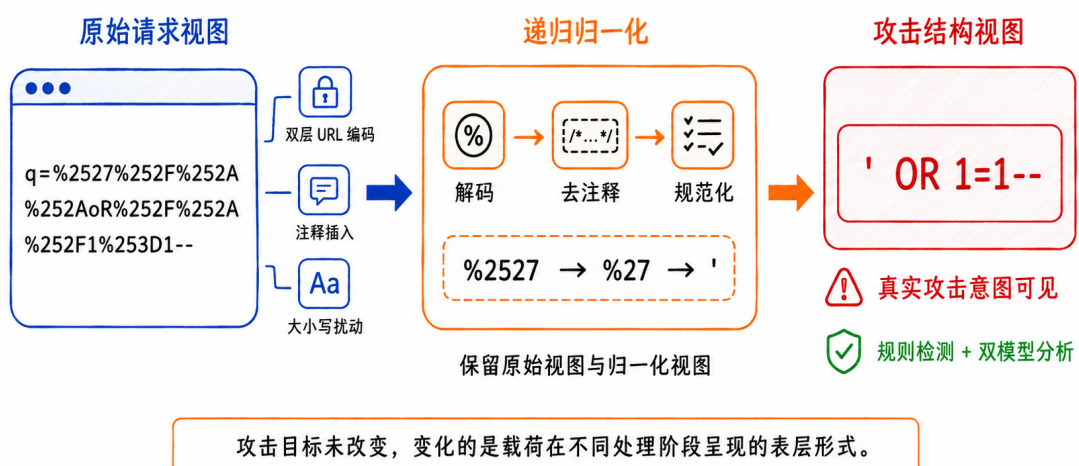


图 1.2 混淆攻击载荷的多视图还原过程

1.2 相关工作分析

1.2.1 基于特征规则的检测方法

规则型 WAF 以关键词、正则表达式和协议异常为主要依据。ModSecurity + OWASP Core Rule Set (CRS) 通过分阶段检查和异常分数累计完成判定，是典型代表 [8, 12]。该路线侧重对已知攻击特征进行快速、可解释的筛查，通常计算开销较低；但实际耗时仍受规则规模和解析复杂度影响。面对编码、拆分和等价变形时，未覆盖的规则可能漏检，简单扩大匹配范围又可能增加误报。

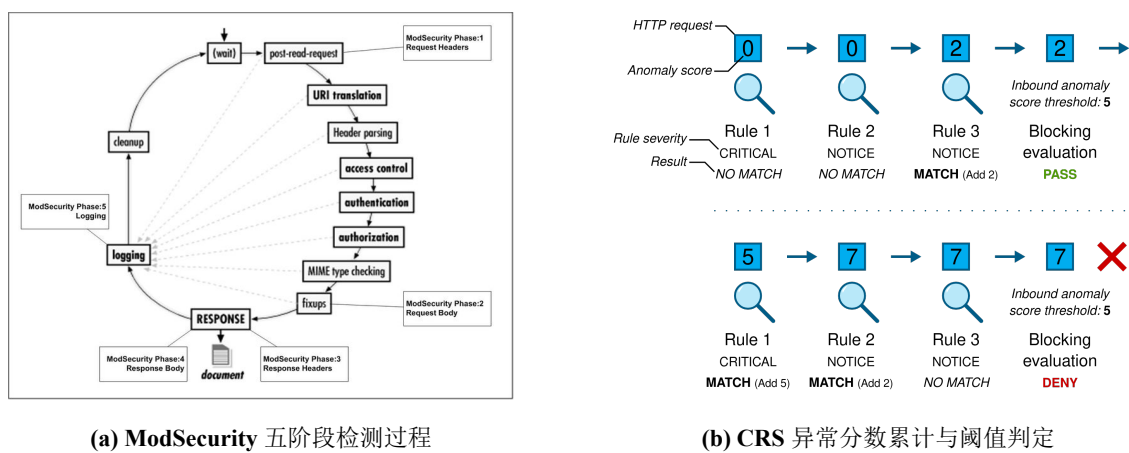


图 1.3 ModSecurity 与 OWASP CRS 的规则检测机制

1.2.2 基于机器学习增强的检测方法

机器学习方法从统计结构、字符片段和上下文中学习攻击模式，可覆盖部分未被规则逐一枚举的变体。LightGBM、TF-IDF 等轻量模型已被用于兼顾效果与开销 [21, 22, 16]；open-appsec 社区版则代表上下文机器学习的工程路线 [14]。该路线侧重利用样本规律扩展未知写法覆盖，但检测效果依赖训练数据与真实业务分布，召回提升也可能伴随误报；特征提取和模型推理还会增加在线耗时，具体幅度取决于模型规模与部署方式。

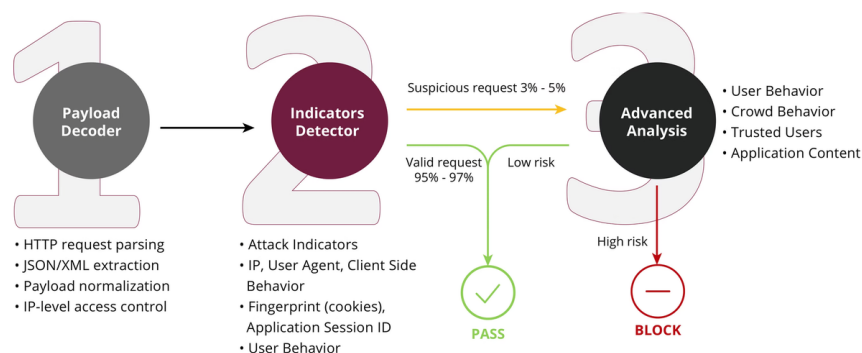


图 1.4 open-appsec 基于上下文机器学习的三阶段检测过程

1.2.3 基于深度解码与语义分析的检测方法

SafeLine 社区版代表基于协议解析和深度解码的产品化规型 WAF，其检测以协议解析、深度解码、语义分析以及成熟的安全规则和工程策略为基础，同时提供反向代理、日志审计和运行管理能力 [15]。本文选择 SafeLine 作为对比基准，是因为它能够代表成熟产品在深层解析、攻击筛选和在线部署上的综合能力。第三章统一测试中，SafeLine 的精确率为 92.4107%、误报率为 0.3696%，体现了其对已覆盖攻击的精细筛选；但召回率和 F1 分别为 57.3861% 和 70.8038%，而 AI-WAF 的混淆攻击召回率达到 98.8946%，对比可用于检验本文方案对混淆变体的识别增益。

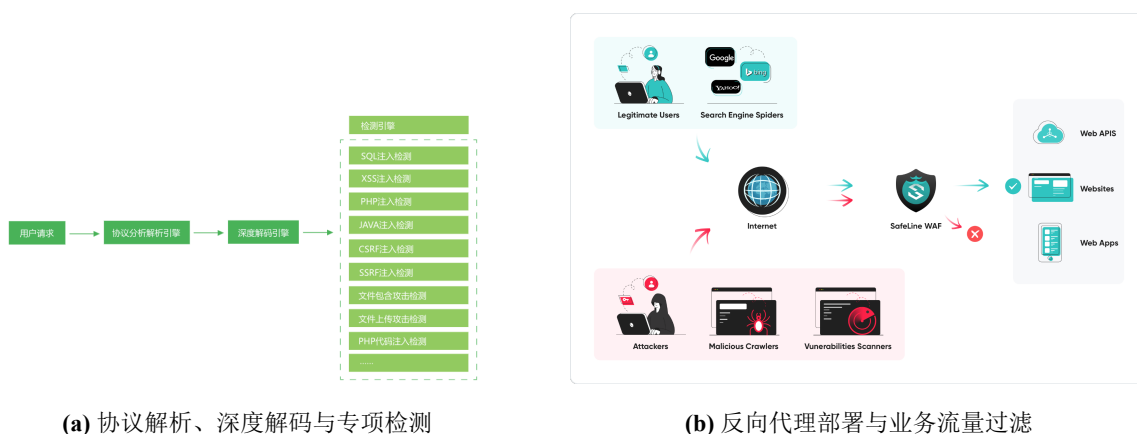


图 1.5 SafeLine 的请求处理流程与部署方式

上述三类方案在检测机制和部署重点上各有侧重，但共同面对三项问题：一是编码与结构变形带来的鲁棒性要求，二是攻击与正常技术文本特征重叠带来的准确性要求，三是完整请求分析、在线时延和稳定转发带来的工程实用性要求。本文因此选取 ModSecurity + CRS、open-appsec 和 SafeLine 的固定版本作为具有不同侧重的基准，并在统一数据与判定口径下比较。

路线	检测与部署侧重	共同约束的具体表现	对比基准
规则型	已知特征、异常评分；路径直接且可解释	混淆覆盖不足；扩展规则可能增加误报与耗时	ModSecurity + CRS
机器学习增强型	从样本和上下文归纳模式，扩展未知写法覆盖	依赖数据分布；特征提取和推理增加在线耗时	open-appsec
产品化规则型	深度解码、语义分析、成熟规则与代理策略	仍受策略覆盖限制；需兼顾误报、时延与业务连续性	SafeLine

表 1.1 三类相关工作的检测侧重、部署重点与共同约束表现

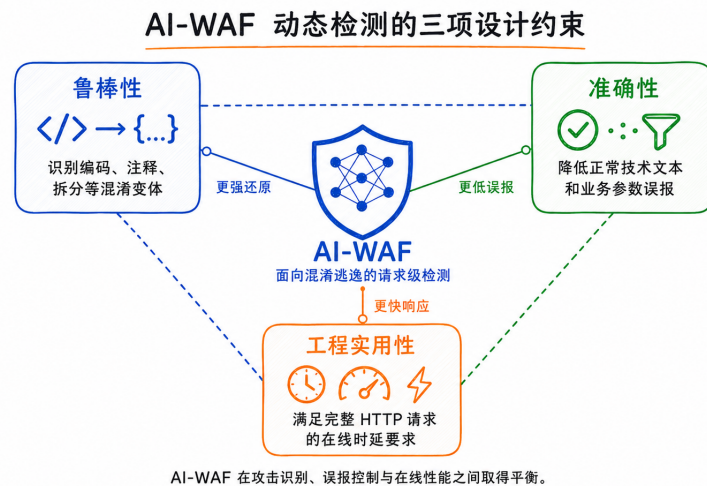


图 1.6 混淆动态检测的三项核心约束

1.3 作品描述

AI-WAF 是一套面向混淆逃逸的 AI 增强型 WAF。它以完整 HTTP 请求为检测对象：先提取路径、参数、请求体、请求头和 Cookie 中的候选载荷并保留字段来源，再通过有边界的递归归一化恢复潜在攻击结构，随后由高置信规则、LightGBM 结构模型和字符 n-gram 文本模型分层判定，最终汇总为请求级结论。系统既支持载荷与请求检测，也可通过前置代理将结论转化为监控或阻断动作。

作品特色集中对应上述三项挑战：

- (1) **以多视图归一化增强鲁棒性。**系统联合检查原始载荷、逐层还原结果和处理过程特征，并覆盖完整请求字段。在 1,357 条混淆攻击上召回率为 98.8946%，与原始

攻击的 98.8014% 基本一致。

- (2) 以规则、双模型和上下文融合保证准确性。规则承担明确证据判断，两个轻量模型分析边界样本，字段上下文与请求级汇总抑制误报。34,721 条独立测试记录上，精确率、召回率和 F1 分别为 99.0476%、98.8515% 和 98.9495%，误报率为 0.0745%。
- (3) 以分层级联和有界处理保障工程实用性。系统限制解码轮次、输入规模与模型复杂度，对明显样本走快速路径；完整载荷链路 P99 延迟为 1.8383 ms。前置代理支持 monitor/block 模式，并已在真实 Web 业务中完成正常请求转发、XSS 请求识别、HTTP 403 阻断和事件留痕。

三项设计共同形成从混淆还原、攻击识别到请求处置的完整检测链路：归一化解解决“看不清”，多源融合解决“判不准”，分层处理解决“来不及”。作品可用于 Web 入口防护，也可作为现有 WAF 的混淆载荷补充分析组件，并在授权、隔离环境中用于规则与模型鲁棒性测试。

第二章 作品设计与实现

系统围绕鲁棒性、准确性和工程实用性三项目标设计，将数据构建、混淆还原、双模型检测、请求级判定和代理防护组织在同一条实际运行链路中，使每项技术方案均对应可验证的系统功能。

2.1 系统架构设计

2.1.1 设计目标

第一章归纳了鲁棒性、准确性和工程实用性三项核心挑战，系统设计目标与其一一对应：

- (1) **识别混淆变体**。解析完整 HTTP 请求，联合检查原始载荷、归一化载荷和处理过程，降低多层编码、注释插入、字符拆分及解析差异造成的漏检。
- (2) **兼顾检出与低误报**。以高置信规则处理明确证据，以 38 维结构模型和字符文本模型分析边界样本，并结合字段上下文形成请求级结论。
- (3) **满足在线处理要求**。限制解码轮次、输入规模和模型复杂度，通过分层快速路径控制开销，将完整载荷链路 P99 延迟不高于 10 ms 作为常态目标，并支持监控与阻断两类请求动作。

2.1.2 总体架构

根据上述目标，AI-WAF 形成离线训练与在线检测两条相互衔接的链路。离线链路完成数据构建、混淆生成、特征构建和双模型训练；在线链路依次完成 HTTP 参数解析、原文检查、递归归一化、规则与双模型判定、请求级汇总及监控/阻断。两条链路共用解析、归一化、特征提取和判定代码，保证训练、评测与在线检测采用一致处理口径。

图 2.1 展示了模块关系及三项设计特性：多视图处理增强混淆鲁棒性，规则与双模型融合提高检测准确性，分层路径和有界处理控制在线开销；字段来源贯穿解析、检测与请求级汇总，使系统能够定位风险位置并保留判断依据。

离线链路为模型提供多来源正常/攻击数据与可追溯混淆变体；在线链路以完整请求为入口，保留字段名称、来源和原始内容，在限定轮次内恢复编码与结构变形，再综合规则、结构特征和字符模式形成载荷级判断。多个字段结果按风险证据汇总为请求结论，并由 API 返回或由前置代理执行相应动作。该架构将数据、模型和请求处

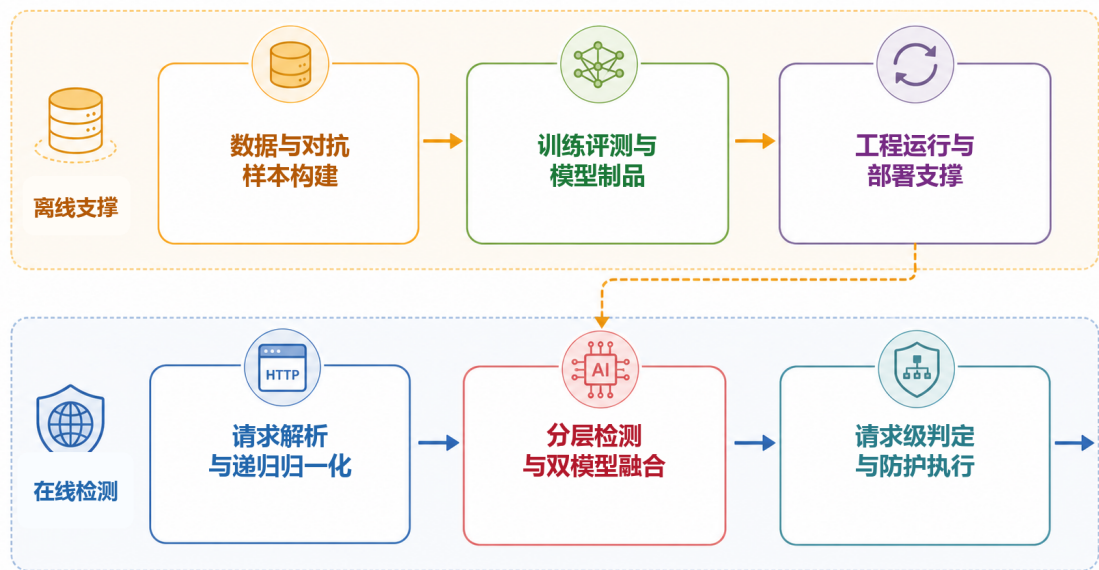


图 2.1 AI-WAF 总体架构及离线、在线模块关系

理组织在同一检测口径下，同时体现鲁棒性、准确性和实时性要求。

2.1.1.3 系统工作流程

在总体架构的基础上，AI-WAF 的工作流程围绕一条完整 HTTP 请求展开。请求从检测 API 或前置代理进入系统后，依次经历请求接入、字段解析、原文初筛、递归归一化、分层检测、双模型融合、请求级汇总和防护执行。各步骤分别由总体架构中的在线模块承担，前一步的输出直接成为后一步的输入。

如图 2.2 所示，流程始终保留字段来源和原始证据。明确样本可在规则或安全边界处提前结束，只有难以确定的载荷进入双模型融合；载荷结论汇总为请求结论后，再由接入方式和运行模式决定最终动作。

请求接入与字段解析。系统首先读取请求方法、URL、请求头和请求体，并检查输入大小与基本协议结构。随后，请求解析模块从路径、查询参数、表单、JSON、请求头和 Cookie 中提取候选载荷。对于嵌套 JSON，系统递归提取叶子节点；对于无法按声明格式解析的请求体，则在安全边界内保留原始正文。每个候选载荷均带有字段名称、来源位置和原始内容。

原文初筛与递归归一化。完成字段提取后，系统先检查原始载荷。已经包含明确攻击结构的内容由高置信规则直接标记，结构简单且无危险迹象的内容可以快速放行。其余载荷进入递归归一化模块，在设定的轮数和长度边界内逐步处理多层编码、

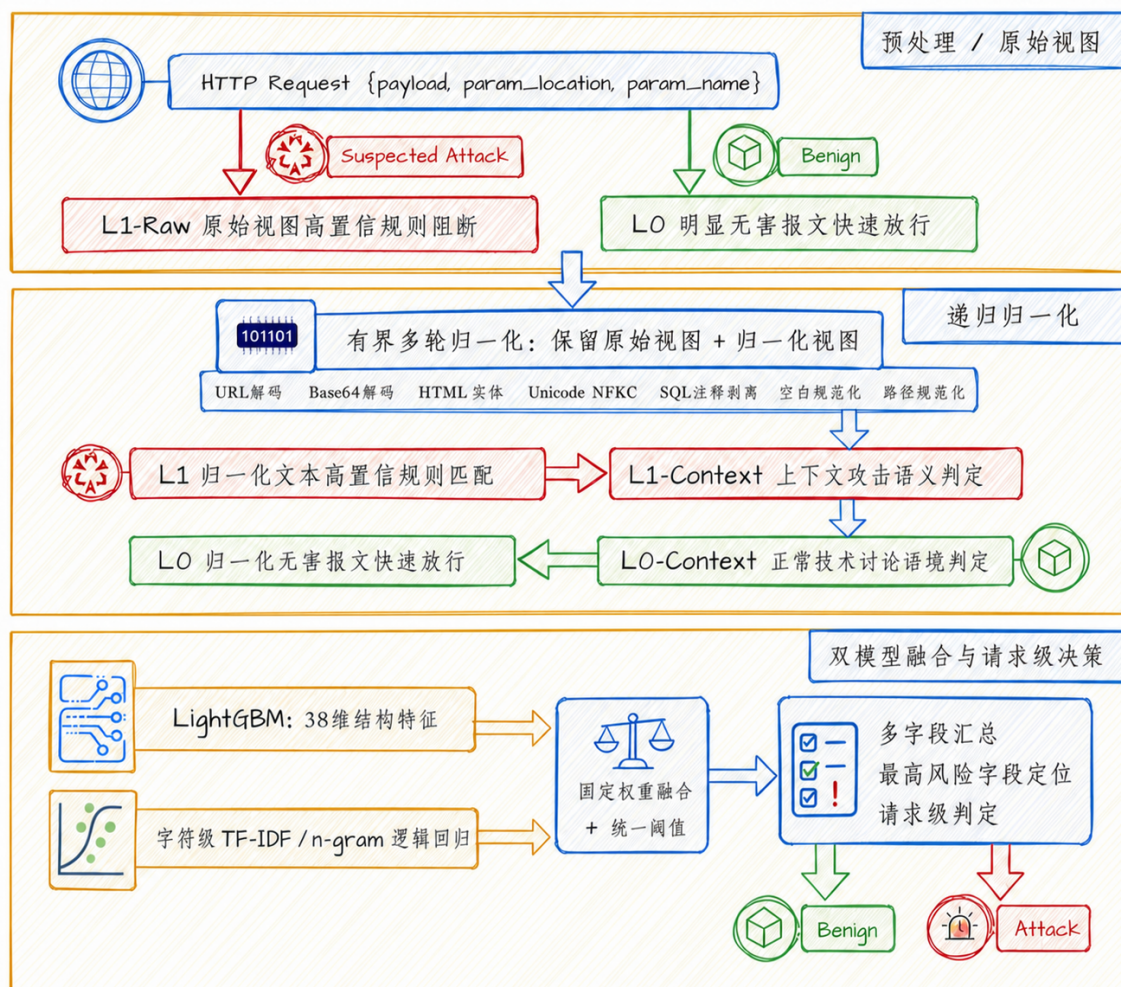


图 2.2 完整 HTTP 请求的在线检测与防护流程

实体转义、字符串拆分和路径变形等混淆，同时记录每一步变化。

分层检测与载荷级判定。归一化完成后，系统同时使用原始载荷、归一化载荷和处理过程信息。规则层继续检查还原后显现的攻击结构，并结合字段名称和来源位置判断其上下文风险。若规则仍不能稳定给出结论，系统将 38 维结构化特征输入 LightGBM 模型，并将原始文本和归一化文本输入字符 n-gram 文本模型。两个分支的概率按照验证集确定的权重和阈值进行融合，形成每条候选载荷的最终结论。

请求级汇总。一条 HTTP 请求可能包含多个候选载荷，系统逐项完成检测后，以风险最高或证据最明确的字段作为请求级判断依据。只要任一关键字段被判定为攻击，整条请求即标记为攻击；全部字段均为正常时，请求才判定为正常。结果同时保留风险字段、来源位置、置信度、检测层级和规则命中信息，便于定位问题。

结果返回与防护执行。如果请求来自检测 API，系统直接向调用方返回结构化结果；如果请求经过前置代理，监控模式会记录结论并继续转发，阻断模式则在攻击请

求到达业务后端前停止转发并返回阻断响应。由此，载荷级判断被进一步转化为完整请求的实际防护动作。

上述流程使用离线阶段确定的模型权重、融合参数和判定阈值，并与训练阶段共用归一化和特征提取逻辑。离线数据与模型为在线检测提供能力，在线流程则把载荷判断落实为完整请求的监控或阻断结论。

2.2 系统实现方案与功能

系统实现由数据与对抗样本构建、请求解析与递归归一化、载荷级双模型检测、多源融合与请求级判定、API 与代理防护五部分组成，各部分按照实际检测链路衔接。

2.2.1 数据集与对抗样本构建

数据是系统识别混淆攻击的基础。如果训练数据只包含少量写法固定的攻击字符串，模型容易记住关键词，却难以识别经过编码、拆分或等价变形后的载荷。为此，本系统围绕“来源多样、类别清晰、变体可追溯、训练与测试相互隔离”组织数据，为后续归一化、特征提取和双模型训练提供统一输入。

多来源样本整理。系统数据主要由公开安全语料、CVE 关联的漏洞信息与利用样例、隔离环境采集数据、项目生成数据和混淆派生数据组成。公开语料包括 PayloadsAllTheThings、fuzzdb 等社区安全测试集合 [24, 25]，用于覆盖常见 Web 攻击类型和典型语法；CVE 关联的公开漏洞信息与对应利用样例补充近期漏洞利用方式、协议级攻击和现代 API 场景，其中漏洞编号与元数据参考 NVD 等公开漏洞数据库 [11]；隔离环境数据保留攻击载荷在完整 HTTP 请求中的位置和结构；项目生成数据覆盖搜索、登录、评论、文件名和 JSON 字段等正常业务内容；混淆派生数据则由原始攻击载荷经过编码、拆分、语法替换和结构变化得到。

原始攻击样本的建设重点不是重复收集少数常见字符串，而是扩大攻击来源、漏洞类型和请求场景的覆盖范围。系统同时整理公开攻击载荷、CVE 漏洞样本、隔离环境中的真实请求结构以及项目构造的协议与上下文样本，使传统 SQL 注入、XSS、命令注入等攻击与反序列化、HTTP 请求走私、SSRF、JWT、API 协议异常等类型共同进入统一数据视图。较全面的原始攻击覆盖为模型学习不同漏洞的共同结构和类型差异提供了基础。

为降低误报，正常数据中还加入了安全教程、代码片段、文件路径和技术讨论等“正常难负例”。这些内容可能包含 SELECT、script 等敏感词，但不具有完整攻击结构，可以帮助模型区分“文本中出现危险词语”和“输入已经形成攻击载荷”。

统一组织与数据口径。不同来源的数据在格式、标签和记录粒度上存在差异。系统对其执行编码统一、标签映射、空值过滤、重复检查和来源记录，并在 data/organized 中形成模型开发所使用的统一数据视图。该目录包含训练、验证、测试及独立评测所需的标准化记录，是模型训练程序读取数据的统一入口，其他原始数据目录主要用于保存上游来源、许可证、原始文件和重建依据，不会在训练时再次重复加入。

统一视图中的记录按照正常或攻击、原始或混淆、攻击类型、数据层级和数据用途进行组织。数据层级包括单字段、完整请求、上下文和协议事件等。正式载荷模型只读取内容非空且明确标记为可用于载荷模型的字段级记录，避免把完整攻击请求的标签错误赋给其中的普通字段。

当前整理后的完整统一数据视图共包含 639,984 条记录，其中正常记录 509,811 条、攻击记录 130,173 条，统一标签覆盖 65 个类别。65 个类别覆盖字段、完整请求、上下文和协议等不同粒度；字段模型训练集覆盖 23 种字段级攻击类型，第三章测试集覆盖其中 22 种。639,984 条表示完整数据资源规模，字段模型实际按照明确的数据层级和用途标记读取训练记录。

训练集、验证集与测试集。符合字段模型使用条件的记录被划分为训练集、验证集和测试集，三者承担不同任务：

表 2.1 训练集、验证集和测试集的作用与数量

数据集合	正常样本	攻击样本	合计	主要作用
训练集	256,856	25,569	282,425	用于训练 LightGBM 特征模型和字符模型，使模型学习正常输入与攻击载荷之间的差异。
验证集	32,228	2,214	34,442	不直接拟合模型参数，用于监控训练过程、选择模型融合权重和攻击判定阈值。
测试集	32,196	2,525	34,721	在模型结构、融合权重和阈值全部确定后进行独立评价，用于计算精确率、召回率、F1 和误报率，不参与训练和调参。

训练集用于学习模型参数，验证集用于选择融合权重和判定阈值，测试集用于评价确定后的完整检测链路。三个集合按照来源组相互隔离，使评价结果能够反映模型对未见样本的识别能力。

验证集中的 2,214 条攻击由 1,013 条原始攻击和 1,201 条混淆攻击组成；测试集中的 2,525 条攻击由 1,168 条原始攻击和 1,357 条混淆攻击组成。原始攻击和混淆攻击同时保留，使系统能够分别评价常规攻击检出能力和面对编码、拆分及等价变形时的鲁棒性。

字段级攻击类型分布。正式字段模型训练范围内共有 23 种攻击类型，覆盖 XSS、SQL 注入、命令注入、路径穿越、SSRF、SSTI、文件包含、XXE 等常见 Web 攻击，也包含开放重定向、反序列化、请求走私、参数污染等需要结合语义或协议结构判断的类型。

训练时保留了全部正常样本和样本较少的攻击类型。XSS 在原始候选训练数据中的数量明显高于其他类别，为防止模型过度偏向单一类型，系统按照原始样本组将其从 12,918 条稳定抽取为 8,000 条；其余 22 种攻击类型的合格训练记录全部保留。验证集和测试集按照既定来源组保持独立。

为直观展示不同攻击类型的总体数量，本文将训练集、验证集和测试集中的攻击样本合并统计，并将数量较少的长尾类别合并为 Other，得到图 2.3。

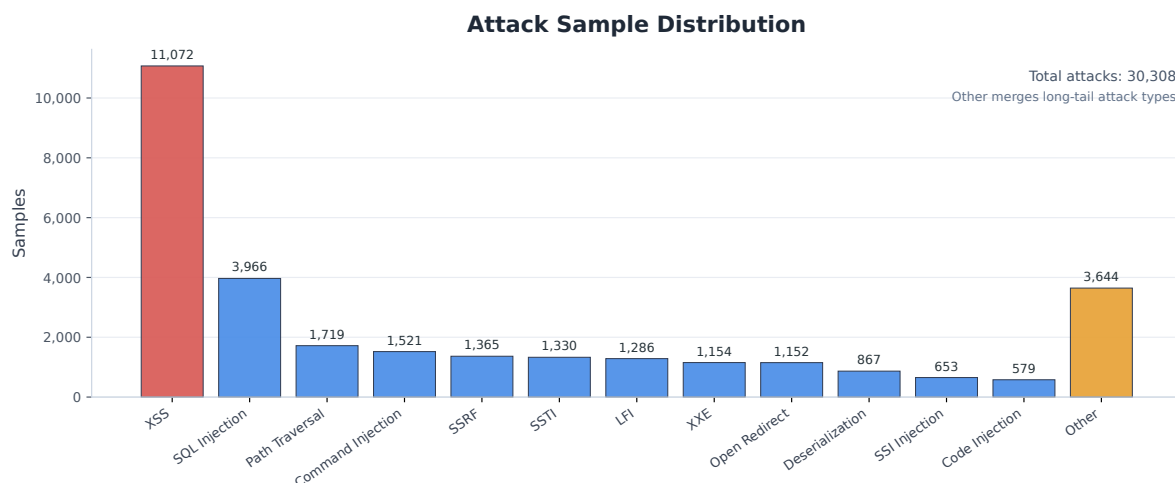


图 2.3 字段模型训练、验证和测试范围内的攻击类型样本分布

图中单独展示样本数量较多的主要攻击类型，并将数量较少的类别合并为 Other，以避免类别过多导致标签拥挤。Other 只是图中的合并展示方式，对应样本仍保留在训练、验证和测试数据中，并不会从模型训练或第三章测试中删除。

部分攻击类型样本较少，主要是因为其有效攻击语法相对集中、可公开审计的来源有限、能够保持攻击语义的变换方式较少，或者检测过程更依赖完整请求与协议上下文。样本数量只表示当前数据覆盖规模，并不直接代表漏洞危害程度。例如 HTTP 请求走私的字段级样本数量较少，但其危害并不低，系统还需要结合完整 HTTP 请求结构进行判断。

开放重定向在训练集中包含 1,152 条样本，当前验证集和测试集中没有对应字段级样本，因此测试集的分类召回率统计覆盖 22 种攻击类型。

数据划分与泄漏控制。数据没有采用简单的逐条随机划分，而是优先按照原始来

源、采集批次、CVE、模板家族和样本组进行切分。同一条原始攻击及其全部混淆变体继承相同的数据分区，防止原始载荷进入训练集、其编码或拆分变体却进入验证集或测试集。

对于项目生成的数据，系统先将同一攻击原型及其正常近邻绑定为一个组，再对组进行确定性划分；对于已经由公开来源规定训练或测试边界的数据，则保留其原有划分。没有明确安全划分的记录只允许进入训练集，不进入验证集和测试集。系统还删除分区内部的精确重复记录，并检查训练、验证和测试之间是否存在相同内容，使第三章的测试结果尽量反映模型对未见样本的识别能力，而不是对近重复字符串的记忆。

分层对抗样本生成机制。本系统的对抗样本生成分为通用混淆接口和面向攻击类型的专项混淆两层。数据构建过程共记录 58 种原子变换，其中 19 种被封装为可直接复用的通用生成接口，其余变换按照不同攻击类型的语义与协议特点组合使用。

第一层为通用生成接口，当前程序注册 19 种可直接调用的编码、结构、等价替换、组合和专项策略，适用于控制台演示、单条载荷测试和一般性数据增强。

表 2.2 通用对抗样本生成接口的 19 种策略

类别	数量	主要策略
编码变换	6	URL 编码、双层 URL 编码、HTML 实体、Base64 编码、十六进制编码和 Unicode 转义。
结构变换	4	SQL 注释插入、JavaScript 注释插入、空白变化和大小写变化。
等价替换	4	SQL CHAR 表示、SQL CONCAT 拼接、SQL 十六进制表示和 JavaScript eval 包装。
组合策略	3	随机组合、递归分层和模拟真实攻击的复合变换。
专项入口	2	路径混淆和命令混淆。
合计	19	通用生成器可以直接调用的注册策略数量。

第二层为面向攻击类型的数据构建策略。系统根据 22 种攻击的语法、路径、协议或运行时特点选择相应变换。专属混淆数据由 2,260 条原始攻击原型派生得到，每条原型生成通用 URL、通用 Base64、类型专项和混合变换等档案，共形成 9,040 条混淆攻击样本，变换链元数据覆盖 58 种不同的原子变换标识。

典型的类型专项变换包括：

表 2.3 不同攻击类型的代表性专项混淆方法

攻击类型	代表性专项变换
SQL 注入	关键字大小写变化、关键字内联注释拆分、空白替换和 MySQL 版本注释。
XSS	标签及关键字大小写变化、控制空白、JavaScript Unicode 转义和 HTML 实体表示。
命令注入	使用 <code>\${IFS}</code> 替代空格、换行分隔和命令名称引号拆分。
路径穿越与 LFI	点号或斜杠百分号编码、双重编码、混合路径分隔符和冗余路径段。
SSRF	IPv4 十六进制表示和 IPv4 十进制整数表示。
HTTP 请求走私	Transfer-Encoding 大小写变化、冒号前空格、Tab 值和重复 TE 请求头。
文件上传	扩展名大小写变化、扩展名前空字节和文件名尾随点。
XXE	XML 实体重命名、引号替换和 XML 结构空白变化。
NoSQL 注入	操作符美元符号 Unicode 化和方括号参数编码。
开放重定向	协议相对 URL、反斜杠、主机尾随点和 URL Scheme 大小写变化。
CRLF 与日志注入	CRLF 混合编码、百分号十六进制大小写变化、双层 URL 编码和 JSON 换行表示。
其他类型	LDAP 过滤字符编码、JWT 填充变化、XPath 空白替换、SSI 指令空白变化、格式化字符串位置参数等。

通用策略解决跨攻击类型共有的编码与结构变化，专项策略则保留特定漏洞的语义约束。例如，SQL 注入使用内联注释拆分关键字，命令注入使用 `${IFS}`、换行或引号拆分，SSRF 改变 IP 地址表示形式，HTTP 请求走私则围绕请求头语法和协议解析差异变换。通用策略与类型专项策略结合，使混淆样本既覆盖共有逃逸方法，也保留不同漏洞的真实攻击语义。

每条派生记录均保留原始样本标识、变换前后内容、攻击类型、变换链、来源组和固定随机种子，用于审计样本来源和复现实验输入。19 种通用接口承担可复用的数据增强，58 种原子变换则反映最终混淆数据中实际覆盖的通用与专项手法。

生成器仅在离线环境中执行字符串变换，不运行生成的攻击载荷，也不向目标服务器发送请求。多来源原始攻击保证漏洞类型和攻击场景覆盖，通用与专项混淆保证逃逸手法覆盖，正常难负例则补充与攻击表面相似的业务文本。三者共同形成覆盖较全面的训练样本，使双模型能够学习攻击结构、字符模式和混淆过程特征，为后续的高准确率、混淆鲁棒性和外部样本泛化能力提供数据基础。

2.2.2 HTTP 请求解析与递归归一化

攻击载荷可能出现在 URL 路径、查询参数、请求头、Cookie 或请求体中，也可能经过多层编码后隐藏原有结构。如果只把完整 HTTP 报文当作一段文本检查，不仅

难以指出具体风险字段，也容易遗漏 JSON 内部或编码内容中的攻击。因此，系统先将请求拆分为可独立检测的候选载荷，再对每条载荷执行原文检查和递归归一化。

本文所说的“载荷归一化”是检测侧的分析过程。系统在内存中生成一个用于判断的归一化副本，不直接修改用户提交的数据；前置代理转发正常请求时仍保留原始方法和请求体，避免检测过程改变业务语义。

HTTP 请求解析。解析器既支持直接提交单条载荷，也支持输入完整 HTTP 请求。对于完整请求，系统依次检查请求行、请求头和请求体，再按照内容类型与字段位置提取候选载荷，主要范围见表 2.4。

表 2.4 HTTP 候选载荷抽取范围

请求区域	抽取对象	处理方式
URL 查询	参数名与参数值	按查询语义解析，保留重复参数及其来源
请求路径	完整路径与路径片段	同时检测整体结构和局部分段
JSON 请求体	叶子键与叶子值	递归遍历对象和数组，保留字段路径
表单请求体	字段名与字段值	按 URL 编码表单规则解析
其他请求体	原始文本	在大小限制和字符解码边界内整体检测
请求头	业务相关头部值	排除常规传输控制头，保留头部名称
Cookie	Cookie 名与值	独立解析并使用更严格的字段阈值
协议结构	长度与传输编码字段	检查报文边界冲突和重复长度字段

对于 JSON 请求体，系统递归遍历对象和数组并提取叶子节点，同时保存类似 `user.filter` 的字段路径；对于 URL 编码表单和 Cookie，则按照键值对分别提取；对于无法按声明类型完整解析的请求体，系统在大小限制内保留原始文本作为候选载荷，并记录解析异常。

解析器不会把所有字段无条件拼接，而是为每条候选载荷保存来源位置、字段名称、原始值和必要的上下文。这样既能定位具体的攻击参数，也能避免不同字段被错误拼接后产生误报。例如，同一个内网地址出现在普通文章中和出现在 `target` 参数中，其安全含义并不完全相同。

归一化前的原文检查。归一化可以恢复攻击结构，也可能改变部分原始证据。例如，编码形式的回车换行在解码后会变成控制字符，某些注释结构可能在清理过程中消失。因此，系统先对原始载荷检查编码控制字符、明确攻击语法和其他高风险结构；原文命中会作为 L1-Raw 证据保留，避免归一化过程形成新的检测盲区。

对于完整 HTTP 报文，系统还会在普通字段检测前检查协议结构。如果请求同时包含 Content-Length 和 Transfer-Encoding，或者出现重复、非法的长度字段，前端代理与后端服务可能对请求边界产生不同理解，系统会将其作为协议异常处理。

多层递归归一化。递归归一化采用有界迭代方式执行。系统首先使用 Unicode NFKC 处理全角等兼容字符，并清理可能用于拆分关键词的零宽字符；随后根据输入形式尝试 URL 编码、HTML 实体、Unicode 与 JSON 转义、十六进制、JavaScript 字符串转义和 Base64 等解码方式，再处理注释插入、字符串拼接、空白变化和路径分隔变形。每轮只接受格式满足要求且结果确实发生变化的变换，并在内容稳定或达到处理上限时停止。

组合混淆往往需要多轮处理。例如，第一次 URL 解码后可能只露出另一层 HTML 实体。系统因此采用“执行一轮、比较结果、继续还原”的方式，最多进行 5 轮外层迭代，其中 URL 解码最多展开 3 层。载荷达到稳定状态、超过轮数上限或不再满足解码条件时，处理立即结束，以避免无限循环和过度计算。

归一化剥离过程如图 2.4 所示。系统不会只保存最终字符串，而是同步提取迭代次数、URL 解码次数、各类解码命中、长度变化、熵变化及可打印字符比例变化。原始载荷和归一化载荷分别进入检测路径，过程特征则加入结构化特征向量。

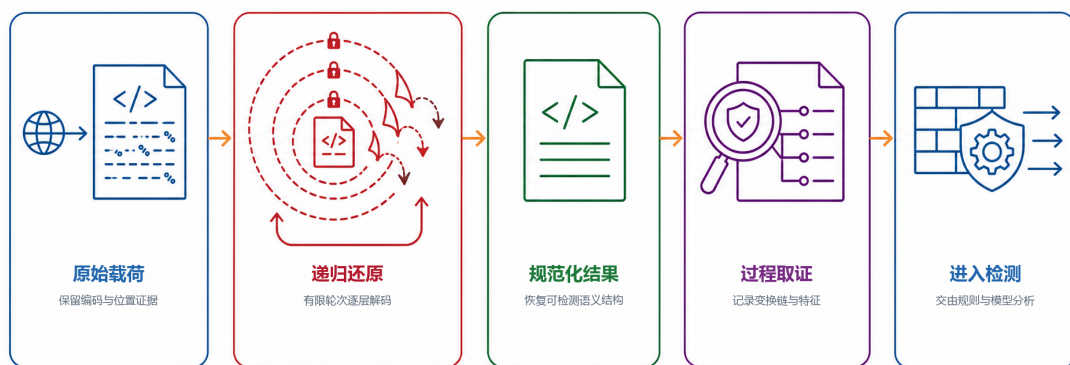


图 2.4 多层 URL 编码载荷的递归归一化与过程特征记录

安全边界与过程记录。系统不会因为字符串符合 Base64 字符集就直接接受解码

结果，而是结合长度、字符集合、可打印字符比例和解码后的结构进行判断；十六进制和转义内容也需要满足相应格式。路径折叠只在路径字段或已经出现路径特征时执行，注释处理也会避免破坏合法 URL。

归一化器最终输出原始载荷、归一化载荷和过程信息三部分。过程信息包括归一化轮数、URL 解码层数、各类解码是否成功、删除的注释与拼接数量、处理前后的长度和熵变化以及是否收敛。这使后续模型不仅能够判断“还原后的内容是什么”，还能够利用“内容经过了怎样的混淆”进行识别。

通过请求解析与递归归一化，系统形成了两项直接能力：一是能够从完整 HTTP 请求中自动定位待检测字段，不要求使用者提前手工拆分参数；二是能够在不改变原始业务请求的前提下，对多层编码和结构变形进行检测侧还原。输出中的字段来源、原始内容、归一化结果和处理过程将继续传递给后续检测模块。

2.2.3 特征提取与双模型分析

完成请求解析和载荷归一化后，系统需要将字符串转换为模型能够处理的输入。仅统计攻击关键词难以覆盖未知变形，只让一个文本模型读取原始内容又可能受到编码层数和随机字符的影响。因此，系统同时构建 38 维结构化安全特征和字符级文本表示，并分别使用 LightGBM 与字符文本模型进行二分类。模型只判断载荷属于正常或攻击，各攻击类型主要用于数据组织和分项评测。

两条模型分支及其融合关系如图 2.5 所示。高置信规则和明确正常输入可提前结束，只有边界样本进入双模型计算；融合结果仍需结合字段位置进行最终判断。

38 维结构化特征。特征提取器对原始载荷计算 12 维特征，对归一化载荷计算 14 维特征，再加入 12 维归一化过程特征，形成

$$\mathbf{x} = [\mathbf{x}_{\text{raw}}^{12}, \mathbf{x}_{\text{norm}}^{14}, \mathbf{x}_{\text{proc}}^{12}] \in \mathbb{R}^{38}. \quad (2.1)$$

三类特征分别保留原始外观、还原后的安全语义和归一化过程信息，并在图 2.5 的结构特征分支中共同输入 LightGBM 模型。

原始层特征用于保留载荷最初的外观，主要描述长度、字符熵、数字与字母比例、特殊字符比例、URL 编码痕迹、HTML 实体、注释结构和不可打印字符等。例如，多层 URL 编码通常会提高百分号编码的比例，即使还原不完整，这些痕迹仍能为模型提供线索。

归一化层特征用于描述还原后出现的安全语义，主要包括 SQL、XSS、命令执行、模板表达式、路径穿越、敏感路径和内网地址等结构，同时考虑引号与括号是否平

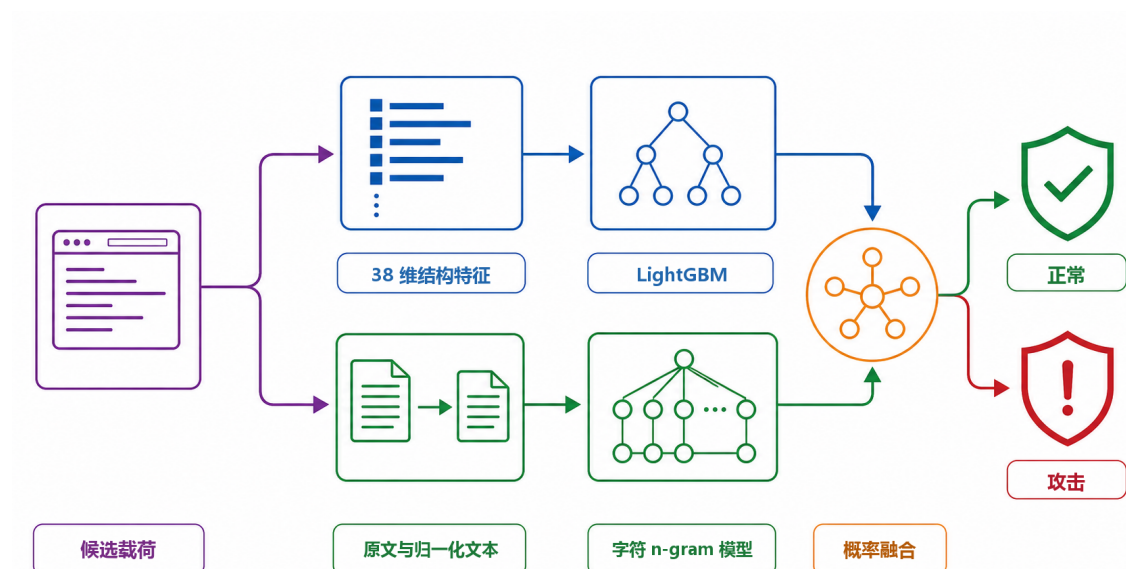


图 2.5 结构特征模型与字符文本模型的互补融合过程

衡、攻击关键词是否集中出现。系统关注的是多个字符和语法片段能否共同形成攻击结构，而不是看到单个关键词就直接判定。

过程层特征来自归一化记录，主要包括迭代深度、URL 解码层数、解码成功次数、注释删除数量、字符串拼接处理数量以及长度和熵的变化。同样的最终文本如果需要经过多层解码才能得到，通常比直接出现的普通文本具有更明显的混淆特征。

三类特征按照固定顺序组成模型向量，训练与在线服务调用同一套提取代码，避免特征顺序不一致造成推理偏差。

LightGBM 特征模型。结构化分支使用 LightGBM 二分类模型。训练参数以 0.02 为学习率、47 个叶节点、最多 1,400 轮提升，行采样和列采样比例均为 0.9， L_1 与 L_2 正则项分别为 0.1 和 2；依据验证集早停，当前模型使用的最佳迭代轮次为 1,241。该模型输出特征分支概率 p_f 。

字符文本模型。文本分支将原始载荷和归一化载荷使用固定分隔标记组织成同一条字符序列，使模型同时看到编码痕迹和还原结果。系统使用 2 至 5 字符的 TF-IDF n-gram 表示，词表上限为 220,000，分类器采用 $C = 6.0$ 的逻辑回归，输出文本分支概率 p_t 。

字符级表示不依赖自然语言分词和完整单词边界。攻击关键词被插入注释、空白或大小写变化后，完整单词可能已经被破坏，但其中仍会保留若干局部字符组合，字符模型能够从训练样本中学习这些模式。与更复杂的深度模型相比，TF-IDF 与逻辑

回归的训练和推理过程更轻量，也更符合在线检测的时延要求。

两个模型关注的信息不同：**LightGBM** 强调字符比例、语法结构和归一化过程，便于解释“哪些结构出现异常”；字符文本模型强调原始与还原文本中的局部顺序，补充人工特征难以列出的变化。模型接口同时返回两个分支概率，并与规则和字段上下文共同用于判定。

由此，系统获得了对单条载荷进行结构与文本双路分析的能力。对于直接攻击，归一化后的危险语法会体现在结构特征中；对于关键词被拆分、大小写被改变或编码未完全还原的载荷，字符模型仍可利用局部字符模式补充判断。两条分支共同服务于载荷级检测，但不会脱离规则和字段上下文单独决定完整请求的处理结果。

2.2.4 多源融合与请求级判定

前述模块分别产生原文规则结果、归一化结果、字段上下文、38 维特征和两个模型概率。系统采用分层级联与概率融合相结合的方式：证据明确的输入先由规则或严格正常判断处理，难以直接确定的输入再进入双模型，从而减少单一方法带来的漏报和误报。

分层级联判断。系统首先检查原始载荷中的高风险结构，命中后保留为 L1-Raw 证据；随后，缺少编码痕迹和攻击结构的数字、布尔值、普通短文本等可以通过 L0 快速路径放行。其余载荷完成归一化后，再依次执行归一化规则、字段上下文规则和正常语义边界检查。

字段上下文用于区分同一内容在不同位置的含意。例如，内网地址出现在普通文章中可能只是说明文字，出现在 `url`、`target` 或 `webhook` 等参数中则需要重点检查；外部 URL 出现在 `redirect` 或 `return_url` 参数中，路径片段出现在 `file` 或 `template` 参数中，也具有不同风险。对于技术文档和论坛讨论，系统只有在高风险规则未命中、字段属于自然语言内容且缺少可执行结构时，才通过 L0-Context 判定为正常。

双模型概率融合。规则和上下文仍无法确定时，系统取得 **LightGBM** 输出的攻击概率 p_f 和字符文本模型输出的攻击概率 p_t ，最终融合分数定义为

$$p = 0.30p_f + 0.70p_t. \quad (2.2)$$

当前统一判定阈值为

$$\tau = 0.35244859, \quad \hat{y} = \begin{cases} 1, & p \geq \tau, \\ 0, & p < \tau. \end{cases} \quad (2.3)$$

其中，0.30 和 0.70 分别为结构特征模型与字符文本模型的权重， τ 为模型攻击判定阈值。这组参数只使用验证集选择，确定后保持固定；测试集不参与调参。模型还保留 0.05 的低风险边界，用于快速确认低分输入。

请求头和 Cookie 中经常包含令牌、会话标识和随机编码字符串，可能表现出高熵或类似 Base64 的形式。为降低这些字段的误报，系统要求其模型融合分数至少达到 0.95 才由模型判定为攻击；如果已经命中控制字符或其他高置信度规则，仍会在规则层直接判定，不受该阈值影响。

检测层级用于描述最终证据来源，当前实现包括 L1-Raw、L0、L1、L1-Context、L0-Context 和 L2+Text。其含义如表 2.5 所示。

表 2.5 载荷级检测层级

层级	主要证据
L1-Raw	原始载荷或编码形式直接命中高置信度攻击规则
L0	缺少编码痕迹和危险结构的明确正常输入快速放行
L1	归一化载荷命中高置信度规则，或规则给出明确结论
L1-Context	字段名称和位置相关的上下文攻击规则命中
L0-Context	确认为不具备执行结构的安全讨论或技术文本
L2+Text	结构化特征与字符文本分支完成融合后形成最终证据

HTTP 请求级汇总。一条完整请求可能包含多个查询参数、请求头、Cookie 和请求体字段。系统先对每条候选载荷独立判断，只要任一字段被判定为攻击，整条请求即被判定为攻击；全部字段均为正常时，请求才被判定为正常。若模型未加载或检测发生异常，系统保留 `uncertain` 状态，在线 API 会返回服务未就绪，而不会把无法完成的检测静默当作正常。

设一个请求抽取得到载荷集合 $\mathcal{R} = \{r_1, r_2, \dots, r_n\}$ ，第 i 条载荷的字段感知概率为 p_i ，则请求风险取

$$p_{\text{req}} = \max_{1 \leq i \leq n} p_i. \quad (2.4)$$

请求级结果同时保存最高风险载荷的来源、字段名称、检测层级、两个模型分支概率和归一化摘要。采用最大风险而不是平均概率，可以避免大量正常字段稀释一个关键攻击字段。对于报文边界冲突等协议异常，系统不再依赖普通字段汇总，而是直接形成请求级风险结论。

通过多源融合和请求级汇总，系统最终形成单载荷检测与完整 HTTP 请求动态检

测两项核心功能。单载荷功能返回正常或攻击结论、置信度、检测层级、规则命中和归一化摘要；完整请求功能则自动检测各候选字段，并指出导致整条请求被判定为攻击的字段及其来源。这样，后续 API 和代理接收的是可直接使用的请求级结论，而不是彼此孤立的模型分数。

2.2.5 API 服务与前置代理防护

完成请求级判定后，系统需要将检测能力提供给外部程序，并接入真实 Web 请求链路。为此，作品实现了检测 API、前置反向代理和运行控制台：API 适合由业务程序或安全平台主动调用，前置代理在不修改原有业务代码的情况下执行监控或阻断，运行控制台则展示代理、模型和安全事件状态。三者共用同一套请求解析、归一化、规则、特征和模型代码，保证检测口径一致。

真实业务接入关系如图 2.6 所示。业务流量只经过对外开放的前置代理；检测服务、正式模型和运行控制台位于内部运行链路，分别负责给出结论、保存事件和调整运行模式。

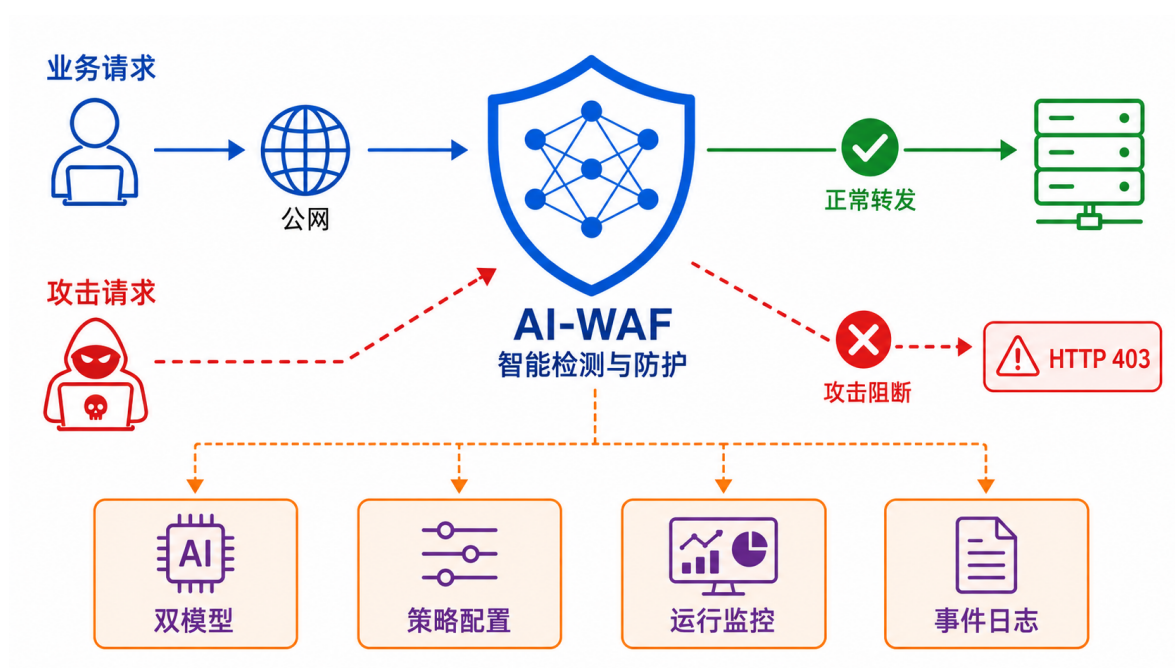


图 2.6 AI-WAF 接入真实 Web 业务的部署与流量关系

检测 API。服务基于 FastAPI 实现，生产运行时只加载检测核心、正式模型和统计模块，不需要加载训练数据与样本生成器。当前公开接口如表 2.6 所示。

表 2.6 AI-WAF 检测服务接口

接口	方法	功能与边界
/health	GET	返回服务和模型加载状态
/detect	POST	检测单条载荷，返回概率、标签、层级和元数据
/batch-detect	POST	批量检测，单次最多 100 条载荷
/detect-http	POST	检测完整 HTTP 请求，默认最大输入为 1 MiB
/stats	GET	查询进程内检测统计
/stats/reset	POST	重置进程内统计数据

单载荷接口接收载荷内容、字段位置和字段名称，载荷长度限制为 1 至 8,192 个字符；批量接口一次最多接收 100 条载荷；完整 HTTP 接口先执行协议检查和字段解析，再返回请求结论与各风险字段。响应中包含正常或攻击结论、置信度、检测层级、规则命中、两个模型分数、归一化摘要和处理耗时，便于调用方执行防护或分析误报。

服务启动时从 `models/current` 加载 LightGBM 模型、字符文本模型和融合配置。任一正式模型缺失或加载失败时，健康检查会显示服务未就绪，检测接口返回 HTTP 503，不使用残缺模型继续提供结果。

前置代理。对于无法修改业务代码或需要在业务之前完成检测的场景，代理位于用户入口与原业务服务器之间。它支持 GET、POST、PUT、PATCH、DELETE、OPTIONS 和 HEAD 等常用方法，可转发至 HTTP 或 HTTPS 后端，并保留正常请求的原始方法与请求体。

代理提供两种检测运行模式：

- **monitor:** 执行检测和审计，发现攻击后记录结果并继续转发，适合首次接入和误报观察；
- **block:** 攻击请求返回 HTTP 403，正常请求转发至后端；

真实业务接入与运行控制。当前系统已经在服务器上接入一套真实 Web 业务，代理将通过检测的请求转发至该业务地址，并在阻断模式下阻止攻击请求进入业务处理程序。运行控制台可以查看代理是否在线、上游业务地址、最后请求时间、正式模型加载状态和近期安全事件，也可以直接切换 monitor 与 block 模式并应用配置。运

行模式只改变攻击请求的后续动作，不改变前面的解析和检测过程。

检测故障处理。运行模式决定“检测到攻击时如何处理”，故障策略则决定“检测程序异常时如何处理”。系统默认采用故障关闭策略，检测异常时返回 HTTP 503；部署者也可以显式选择故障开放，在已有其他防护且业务可用性优先时记录异常并继续转发。业务后端无法连接时返回 HTTP 502，以便与检测服务异常区分。

协议检查与安全转发。代理在检测前检查传输编码和内容长度字段，拒绝二者同时出现的请求以及重复或非法的长度字段，并按照协议检查分块传输格式。请求体默认上限为 1 MiB，避免异常大请求持续占用资源。

转发时，代理移除只对当前连接有效的逐跳请求头，重新设置必要的主机、长度和可信转发信息，避免客户端伪造内部代理标识。路径名称本身不会因为包含 `admin`、`shell` 等单词就直接触发阻断，生产防护重点仍是用户可控参数、请求头、Cookie 和请求体，以减少正常业务路由造成的误报。

日志与运行状态。审计日志采用 JSON Lines 格式，记录请求编号、时间、方法、路径摘要、运行模式、检测结论、风险字段、攻击证据、代理链路耗时和故障类型。为减少敏感信息泄露，日志不保存完整查询值、Cookie 和请求体正文。运行控制台在此基础上汇总请求总量、攻击与阻断数量、变化趋势和近期安全事件，使管理人员可以从请求记录进一步定位攻击字段和命中依据。

控制台展示的是代理链路的整体处理耗时，而不是模型单独完成一次判断所需的时间。该时间覆盖请求接收、HTTP 解析、字段提取、归一化、检测和日志记录；正常请求还包含上游转发与响应过程，并会受到服务器性能、网络状态和业务页面资源请求数量影响。模型自身的检测耗时因此在第三章使用独立载荷级测试单独评价，避免将两种时间口径混为一谈。

通过上述实现，系统形成了单载荷、批量载荷和完整 HTTP 请求三类在线检测接口，并具备无需修改业务代码的前置代理防护能力。API 负责向外部程序提供结构化结果，代理把请求级结论转化为记录、转发、返回 403 或检测故障处理等实际动作，运行控制台则提供状态查看、事件追踪和模式切换。三者共同使前述检测方案进入真实 Web 请求链路。

第三章 作品测试与分析

3.1 测试方案

评测以原文规则、递归归一化、上下文判断和双模型融合组成的完整载荷检测链路为主要对象，并补充组件消融、固定版本 WAF 对比、完整请求及在线运行测试。训练集仅用于模型学习，验证集用于早停、融合权重与阈值选择，参数确定后，独立测试集只用于最终统计。同一原始载荷及其混淆变体按组划入同一分区，避免相近样本跨训练集与测试集造成数据泄漏。

测试围绕准确性、鲁棒性和工程实用性三项设计目标展开，同时验证组件贡献、横向竞争力和请求级能力，具体安排见表 3.1。

表 3.1 第三章测试目标与评价内容

测试目标	测试对象	主要评价内容
准确性	34,721 条正式测试记录	精确率、召回率、F1、误报率与混淆矩阵
鲁棒性	原始攻击与混淆攻击	混淆前后召回率及总体攻击检出情况
组件贡献	六组规则、归一化与模型组合	各组件增益及检测质量-延迟取舍
横向对比	三个固定版本 WAF 产品	统一数据和判定口径下的效果与开销
请求级能力	正常上下文、编码载荷与协议异常	风险字段、还原证据和请求级结论
工程实用性	完整检测链路与前置代理	P99 延迟及监控、转发和阻断动作

主要指标如表 3.2 所示。F1 为精确率与召回率的调和平均数，可综合反映漏报与误报的平衡 [23]；P99 用于描述常态高分位时延。

表 3.2 主要评价指标及其含义

指标	定义与用途
精确率	$TP/(TP + FP)$ ，衡量攻击判定的可信程度。
召回率	$TP/(TP + FN)$ ，衡量真实攻击的检出能力。
F1	$2PR/(P + R)$ ，综合评价精确率与召回率。
误报率	$FP/(FP + TN)$ ，衡量正常流量被误判的比例。
P95/P99	95%/99% 记录不超过的检测时延，反映高分位性能。
吞吐量	单位时间完成的检测记录数，反映批量处理能力。

正式字段级测试集共 34,721 条，组成如表 3.3 所示。该测试集不参与训练、融合

权重选择或阈值确定；横向测试也使用同一批记录和统一判定口径。

表 3.3 正式测试集组成

记录类型	数量	占比
正常记录	32,196	92.73%
原始攻击记录	1,168	3.36%
混淆攻击记录	1,357	3.91%
合计	34,721	100.00%

3.2 载荷检测与对抗能力测试

图 3.1 先给出完整检测链路的总体结果。34,721 条正式测试记录上的精确率为 99.0476%，召回率为 98.8515%，F1 为 98.9495%，误报率为 0.0745%。四项指标共同说明：系统在保持较高攻击检出能力的同时，对正常业务流量造成的误报较少。

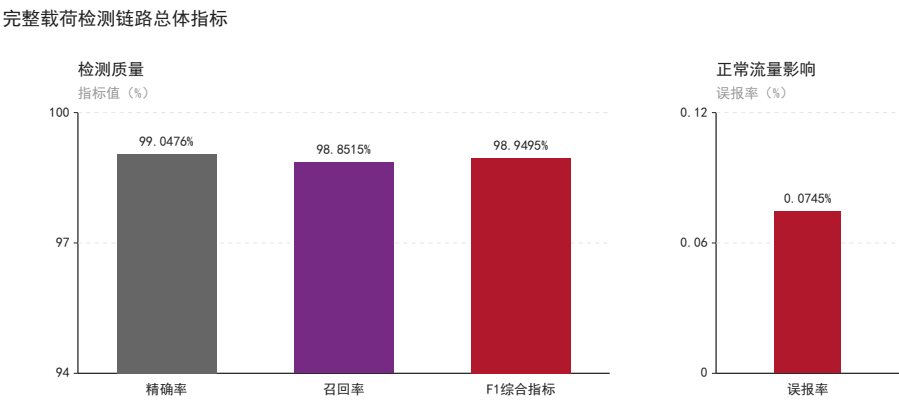


图 3.1 完整载荷检测链路的总体检测指标

为便于理解总体指标的具体构成，图 3.2a 展示了测试集上的混淆矩阵。32,196 条正常记录中有 32,172 条被正确放行，仅 24 条被误判为攻击；2,525 条攻击记录中有 2,496 条被检出，仅漏检 29 条。误报和漏报数量均较少，与图 3.1 的结果相互印证。混淆攻击是本文重点面对的问题，图 3.2b 对比了原始攻击、混淆攻击和全部攻击的召回率。由总混淆矩阵与混淆攻击分项可得，系统对 1,168 条原始攻击检出 1,154 条，召回率为 98.8014%；对 1,357 条混淆攻击检出 1,342 条，召回率为 98.8946%；全部

2,525 条攻击的召回率为 98.8515%。三者处于同一水平，说明递归归一化、过程特征和字符文本模型能够削弱编码嵌套、注释插入、大小写扰动和字符串拆分带来的逃逸效果。

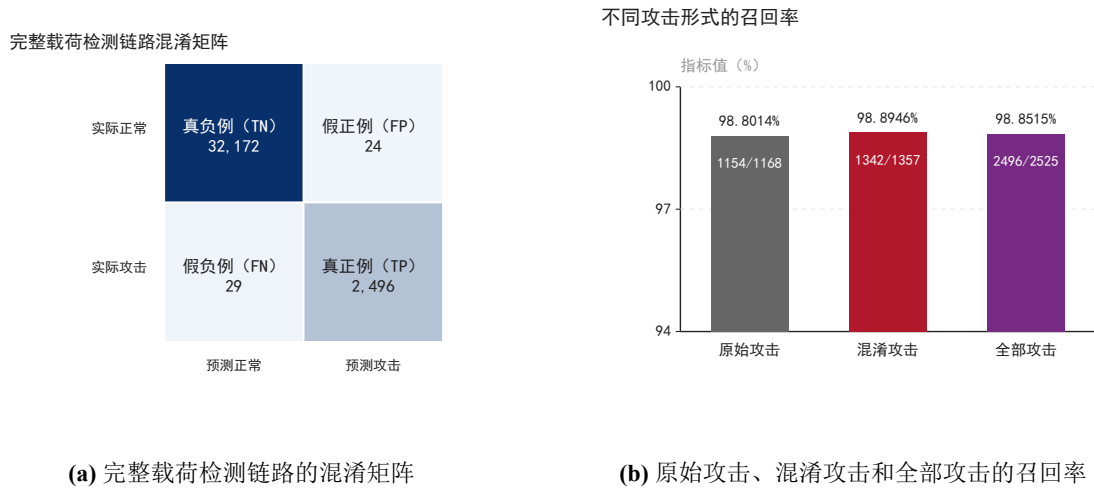


图 3.2 完整载荷检测链路的准确性与混淆鲁棒性表现

3.3 组件消融与性能对比

为分析规则检测、递归归一化和双模型对最终结果的贡献，本实验设置“仅规则”“规则+归一化”“仅 LightGBM”“仅字符模型”“双模型融合”和“完整链路”六种方案，并在相同测试集和评价口径下比较检测效果与处理时延。

测试集共包含 9,500 条流量，其中正常流量 8,500 条，占 89.4737%；攻击流量 1,000 条，占 10.5263%。正常流量占比较高，用于模拟实际业务中正常请求占主体的流量结构，使误报率及时延的测试结果更接近系统上线后的关注重点。该比例用于构造具有业务特征的受控测试环境，不代表所有生产系统均具有相同的流量分布。

表 3.4 组件消融测试集的流量组成

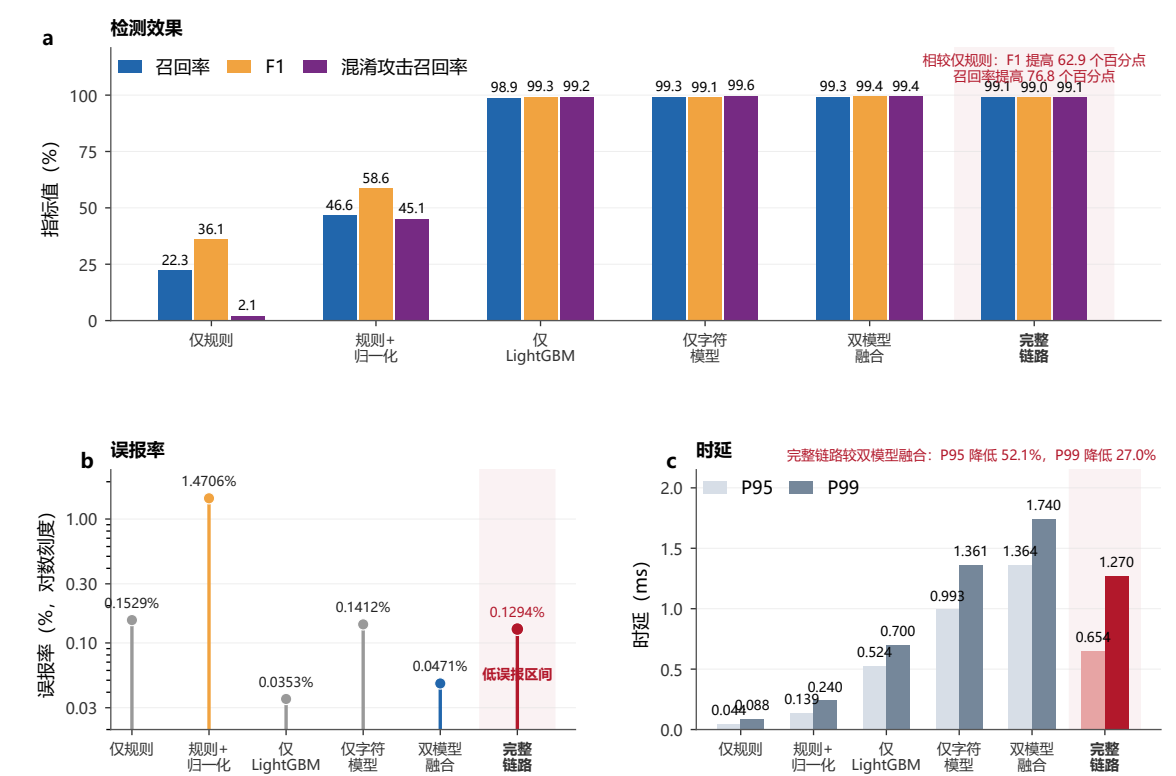
流量类型	数量	占比
正常流量	8,500	89.4737%
攻击流量	1,000	10.5263%
合计	9,500	100%

完整链路采用分层处理方式。当前配置下，9,025 条流量在前置路径完成判定，占测试集的 95%，其中包括全部 8,500 条正常流量和 525 条攻击流量；其余 475 条攻击

流量进入双模型检测。各方案采用 P95 和 P99 衡量处理时延。将所有样本的处理时间从小到大排列，P95 表示约 95% 的样本可在该时间内完成检测，P99 表示约 99% 的样本可在该时间内完成检测。二者分别反映系统的常态高分位时延和更靠近最慢端的高分位时延。

表 3.5 六组组件消融与性能对比结果

方案	精确率	召回率	F1	误报率	混淆攻击召回率	P95/ms	P99/ms
仅规则	94.4915%	22.3000%	36.0841%	0.1529%	2.0755%	0.0444	0.0885
规则+归一化	78.8494%	46.6000%	58.5795%	1.4706%	45.0943%	0.1386	0.2401
仅 LightGBM	99.6976%	98.9000%	99.2972%	0.0353%	99.2453%	0.5245	0.7002
仅字符模型	98.8060%	99.3000%	99.0524%	0.1412%	99.6226%	0.9931	1.3609
双模型融合	99.5988%	99.3000%	99.4492%	0.0471%	99.4340%	1.3639	1.7405
完整链路	98.9022%	99.1000%	99.0010%	0.1294%	99.0566%	0.6537	1.2704



固定测试集：9,500 条流量，其中正常流量 8,500 条（89.4737%）；完整链路前置路径完成 95% 流量的判定。

图 3.3 六组方案的检测效果、误报率与 P95/P99 时延对比

实验结果表明，规则检测具有明显的速度优势，仅规则方案的 P95 和 P99 分别为 0.0444 ms 和 0.0885 ms，适合快速识别具有明确特征的攻击。然而，该方案的整体召回率仅为 22.3000%，混淆攻击召回率仅为 2.0755%，说明固定规则难以覆盖编码、拆

分和等价改写后的攻击载荷，不能单独承担完整检测任务。

加入递归归一化后，整体召回率由 22.3000% 提升至 46.6000%，混淆攻击召回率由 2.0755% 提升至 45.0943%，分别提高 24.3000 和 43.0188 个百分点，证明归一化能够有效恢复被编码或结构变换隐藏的攻击特征。但该方案的误报率上升至 1.4706%，精确率下降至 78.8494%。这说明归一化扩大了规则可见范围，同时也可能使正常编码内容呈现出与攻击相似的局部特征，因此归一化适合作为特征恢复手段，不能脱离上下文和模型判断直接作为最终结论。

两个学习模型表现出不同侧重。LightGBM 的精确率为 99.6976%，误报率仅为 0.0353%，P95 为 0.5245 ms，在检测准确性和处理速度之间取得了较好平衡；字符模型的整体召回率和混淆攻击召回率分别达到 99.3000% 和 99.6226%，对字符拆分、局部变形和混淆语法更加敏感，但其 P95 和误报率均高于 LightGBM。双模型融合后，F1 提升至六种方案中的最高值 99.4492%，同时将误报率控制在 0.0471%，说明结构特征和字符模式具有互补作用；相应地，同时执行两个模型使其 P95 和 P99 增加至 1.3639 ms 和 1.7405 ms。

完整链路的精确率、召回率和 F1 分别为 98.9022%、99.1000% 和 99.0010%，误报率为 0.1294%。换算攻击样本数量，系统正确检出 1,000 条攻击中的 991 条，仅漏检 9 条；在 8,500 条正常流量上的影响仍以误报率评价。其混淆攻击召回率达到 99.0566%，说明分层处理没有明显削弱对混淆攻击的识别能力。

与始终执行双模型的方案相比，完整链路的 F1 降低 0.4482 个百分点，但 P95 从 1.3639 ms 降至 0.6537 ms，降低约 52.1%；P99 从 1.7405 ms 降至 1.2704 ms，降低约 27.0%。产生这一结果的主要原因是前置路径承担了 95% 流量的判定，仅将少量难以确定的攻击载荷交给双模型处理。P95 反映了大多数请求经过快速路径时的常态开销，P99 则体现复杂样本进入模型路径后的高分位时延。

综合来看，规则路径速度快但对混淆变体覆盖不足，归一化能够恢复隐藏结构但不能独立解决误报问题，LightGBM 与字符模型分别在效率和混淆召回方面具有优势，双模型融合取得最高检测指标，而完整链路通过分层调度在仅小幅牺牲 F1 的情况下显著降低了时延。因此，完整链路的主要价值不是在每一项单独指标上均达到最高值，而是在正常流量占主体的业务场景下，同时保持较高的攻击检出率、较低的误报率和可控的在线处理时延。

3.4 与已有 WAF 产品的对比测试

本文选取 ModSecurity + OWASP CRS、open-appsec 和 SafeLine 三个实际 WAF 产品作为参照。三者分别对应第一章中讨论的典型技术路线：ModSecurity + CRS 代表传统规则匹配型 WAF，open-appsec 代表机器学习增强型 WAF，SafeLine 代表以成熟规则与工程策略为基础、面向真实业务部署的集成式 WAF 产品。对比对象及实验身份如表 3.6 所示。

表 3.6 WAF 对比产品与技术路线

方案	技术路线	实验定位
ModSecurity 3.0.16 + CRS 4.28.0	规则匹配型 WAF	规则清晰、运行开销较低，适合作为传统规则路线基准；能力取决于规则库对攻击写法的覆盖。
open-appsec 1.1.35-open-source	机器学习增强型 WAF	试图提升对未知或变形攻击的覆盖能力，用于观察学习型防护在召回率、误报率和耗时之间的取舍。
SafeLine CE 9.3.11	集成式业务 WAF 产品	以真实反向代理栈部署，代表可直接用于业务入口的成熟社区版产品。
AI-WAF	规则 + 归一化 + 双模型融合	本文方案，完整链路包含原文规则、递归归一化、上下文边界和双模型融合。

AI-WAF 与三个对比产品在同一批完整实验中处理相同的 34,721 条 organized 测试记录，记录标签、输入顺序和重复排除规则保持一致。每条字段记录按照其位置转换为 query、body、header、cookie、path 或 multipart filename 等真实 HTTP 请求位置；只有产品明确返回受控阻断且后端没有到达证明时，才记为检出。5xx、超时和连接异常不计为成功检出。三个对比产品均使用固定版本和固定策略。

表 3.7 同一批实验产出的 WAF 检测指标与延迟

方案	精确率	召回率	F1 综合指标	误报率	接入 P99/ms
AI-WAF	99.0476%	98.8515%	98.9495%	0.0745%	2.2567
ModSecurity 3.0.16 + OWASP CRS 4.28.0	40.1990%	52.7921%	45.6429%	6.1592%	3.9064
open-appsec 1.1.35-open-source	29.9154%	77.0693%	43.1008%	14.1601%	27.5188
SafeLine CE 9.3.11	92.4107%	57.3861%	70.8038%	0.3696%	16.2656

从图 3.4 和表 3.7 可以看出，三组对比从不同侧面体现了本文系统的优势。

与 ModSecurity + CRS 对比，体现检测准确性优势。ModSecurity + CRS 采用传

本文系统与三个已有 WAF 产品的同一测试集检测指标对比

四个方案使用相同的 34,721 条独立测试记录

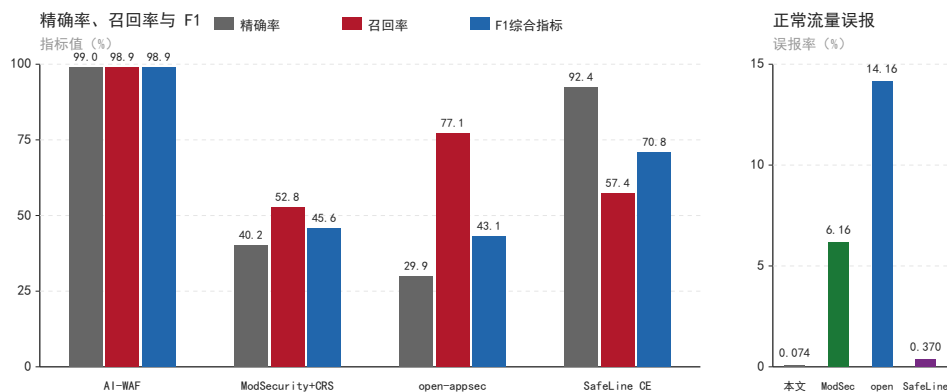


图 3.4 AI-WAF 与三个已有 WAF 产品的检测指标和误报率对比

统规则匹配路线，判定过程清晰且运行开销较低，但其精确率、召回率和 F1 分别为 40.1990%、52.7921% 和 45.6429%，对规则未覆盖的攻击写法和混淆变体适应能力有限。AI-WAF 的精确率、召回率和 F1 均接近 99%，说明在保留低延迟的同时，规则初筛、递归归一化和双模型融合显著提高了攻击判断的准确性。

与 open-appsec 对比，体现检测效率与数据覆盖优势。open-appsec 作为机器学习增强型 WAF，召回率达到 77.0693%，高于传统规则基准，说明学习型方法能够扩大攻击覆盖；但其精确率为 29.9154%、F1 为 43.1008%、误报率为 14.1601%，且本次完整实验中的接入 P99 为 27.5188 ms。AI-WAF 的接入 P99 为 2.2567 ms，同时取得 98.9495% 的 F1 和 0.0745% 的误报率。该差异说明轻量双模型与分层级联机制降低了每条载荷的计算开销，而多来源原始攻击、CVE 与新型漏洞流量、通用混淆和类型专项混淆共同形成的全面训练样本，使模型在扩大覆盖时仍能保持较高精确率和较低误报。

与 SafeLine 对比，体现混淆攻击识别优势。SafeLine 的精确率为 92.4107%、误报率为 0.3696%，表明其成熟规则和工程策略对已覆盖攻击具有较精细的筛选能力；但其召回率为 57.3861%、F1 为 70.8038%，仍低于 AI-WAF。AI-WAF 对混淆攻击的召回率达到 98.8946%，说明模型能够从全面训练样本中学习跨写法的攻击结构，而不局限于已经枚举的规则形式。SafeLine 与 AI-WAF 的运行形态和宿主机 CPU 利用方式并不完全相同，且未进行专项调优，因此 16.2656 ms 的接入 P99 反映当前测试环境中的实测开销，不代表其生产环境最优性能。

上述横向对比均通过外部反向代理、HTTP 传输和后端到达证明完成判定，表 3.7

中的接入 P99 用于比较当前部署形态下的整体开销；后文 1.8383 ms 的载荷级 P99 则来自进程内字段消融计时，两者范围不同，不能互换。按统一的接入口径观察，AI-WAF 仍同时保持较高检测指标和较低接入延迟。

综合三组对比，AI-WAF 在同一批次中正确识别 2,496 条攻击，仅漏检 29 条；32,196 条正常记录中仅有 24 条被误报。优势来源可以概括为：多来源原始攻击与 CVE、新型漏洞流量保证攻击类型覆盖，19 种通用接口与 58 种原子变换构成的通用加专项混淆保证逃逸手法覆盖，正常难负例补充业务边界；由此形成覆盖全面的训练样本，再由规则快速路径、递归归一化和双模型共同完成判断。最终模型表现为覆盖较完整、判断准确，并具备较好的混淆鲁棒性和在线处理效率，与第二章的数据集构建和系统设计形成对应。

3.5 HTTP 请求级动态检测测试

字段级记录用于稳定比较检测效果，在线检测服务则用于验证完整 HTTP 请求解析、风险字段定位和请求级汇总能力。测试覆盖 URL 编码攻击、正常安全讨论、活动函数调用和协议结构异常，所用模型与离线测试完全一致。

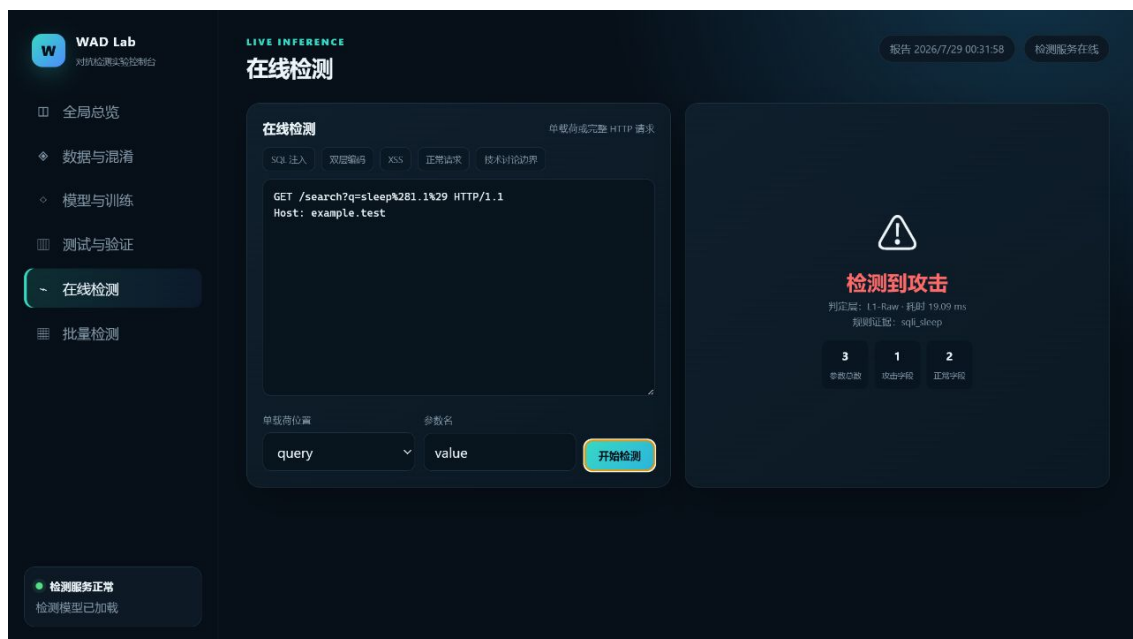
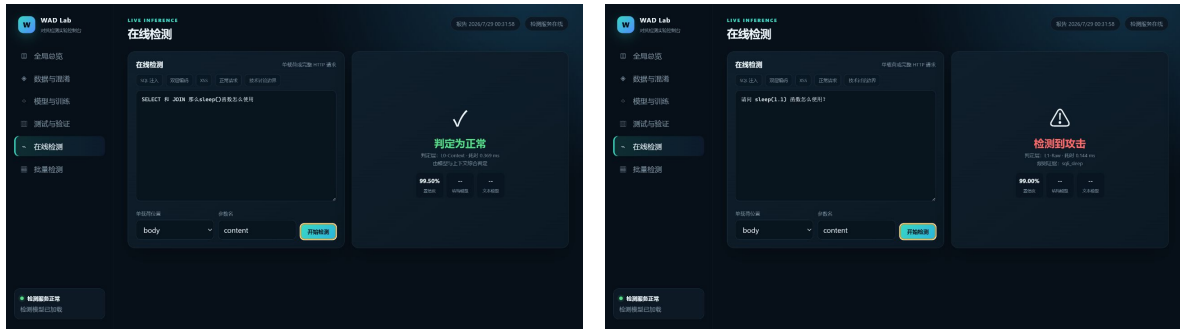


图 3.5 URL 编码攻击的字段定位与请求级判定

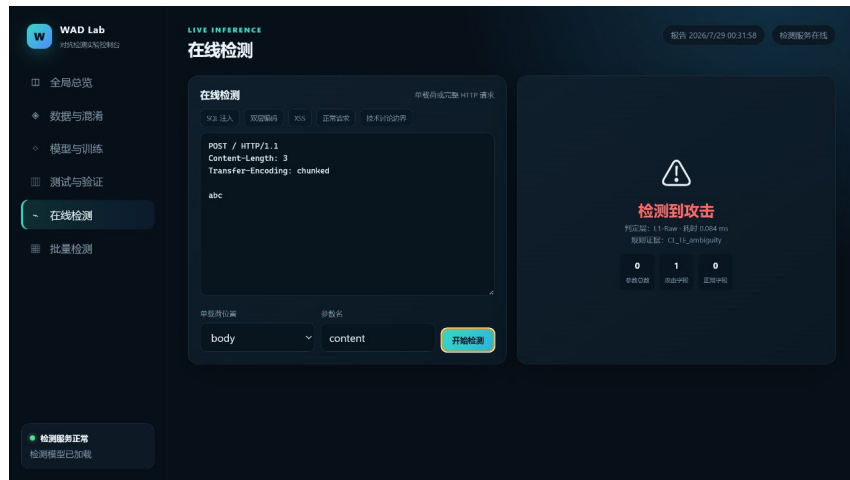
图 3.5 展示 URL 编码攻击的检测结果。系统从完整请求中提取查询参数 q ，将编码内容还原为 `sleep(1.1)`，并把该字段及整个请求判定为攻击。页面同时给出字段位置 `query`、检测层级 `L1-Raw` 和规则依据 `sql_sleep`，说明检测结论可以回溯到具

体字段和具体证据。



(a) 正常安全讨论被放行

(b) 活动延时函数被识别



(c) 协议字段冲突被直接拦截

图 3.6 正常语义、活动攻击与协议异常的检测对照

图 3.6 给出三组对照。包含 SELECT、JOIN 和 sleep() 的安全讨论文本命中上下文正常边界，被判定为正常；带有可执行延时参数的 sleep(1.1) 被判定为攻击；同时出现 Content-Length 与 Transfer-Encoding 的报文命中协议歧义规则，直接形成请求级攻击结论。由此可见，系统不是简单查找危险关键词，而是结合字段位置、文本语义、归一化结果和协议上下文完成请求级判断。

3.6 在线防护与运行性能测试

3.6.1 载荷级检测性能

实时性是 WAF 能否接入业务链路的重要条件。图 3.7 展示了载荷级完整检测链路的延迟：平均延迟为 0.7933 ms，P99 为 1.8383 ms，按逐记录计时总和折算的吞吐量为 1,260.481 records/s，均明显低于第一章提出的 10 ms 常态目标。P99 表示 99% 的字段载荷能在该时间内完成预处理和推理，比最大值更能反映常态性能。产品接入链

路的 P99 已列在表 3.7 中，其范围还包含反向代理、HTTP 传输和后端到达证明，因此高于本处的进程内载荷级结果。

载荷级检测延迟与性能目标

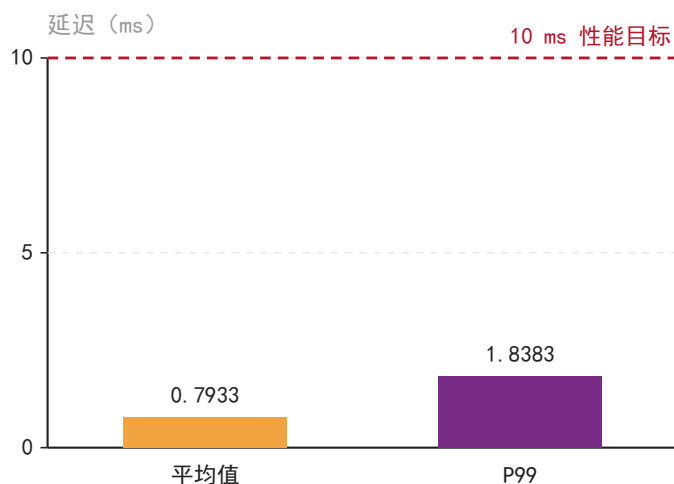
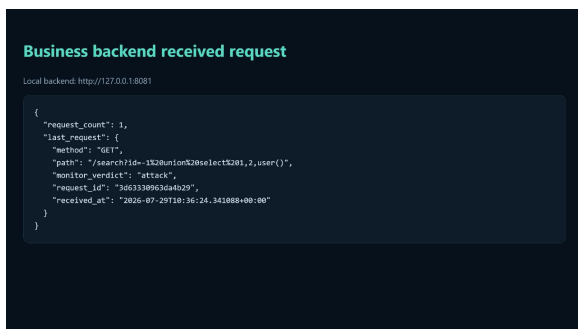


图 3.7 完整载荷检测链路的延迟与性能目标

3.6.2 代理模式与端到端动作验证

前置代理测试进一步验证了检测结论能够转化为实际防护动作。同一 SQL 注入请求在 monitor 模式下被识别并记录为 `monitored_attack`，但仍被转发，后端计数增加；切换到 block 模式后，相同请求返回 403，日志记录为 `blocked`，后端计数不再增加。正常请求在两种模式下均返回 200 并被转发。图 3.8 形成了“检测结论—代理动作—审计日志”的完整证据链，也说明系统可以先以监控模式观察业务，再按需要切换到阻断模式。



(a) monitor: 识别攻击并继续转发



(b) block: 攻击请求返回 403



(c) 代理日志记录检测结论与转发动作

图 3.8 monitor 与 block 模式的端到端运行结果

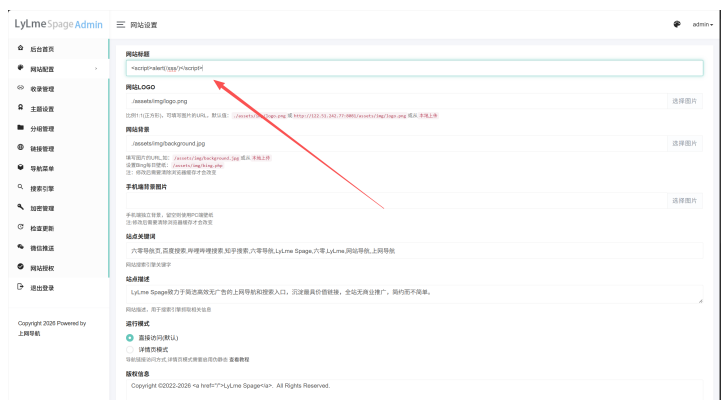
3.6.3 真实业务 XSS 防护验证

为进一步验证系统在实际业务中的运行效果，本作品在服务器上部署了一套可正常访问和管理的 Web 应用，并将 AI-WAF 以前置代理方式部署在业务服务之前。外部请求首先进入 AI-WAF，完成 HTTP 解析、字段提取和攻击检测后，再根据当前模式决定转发或阻断。图 3.9 展示了接入代理后的真实业务管理后台，该环境包含实际页面、后台设置、数据库和静态资源请求，不是仅返回固定结果的模拟接口。



图 3.9 接入 AI-WAF 前置代理的真实业务管理后台

在授权测试过程中，测试人员在后台“网站标题”字段中填写典型 XSS 测试载荷并提交。代理从表单请求体中提取该字段，识别出脚本标签和可执行函数等攻击特征，并将完整请求判定为攻击。在 block 模式下，请求在进入上游业务处理程序前被停止转发，客户端直接收到 HTTP 403 阻断页面，如图 3.10 所示。恶意内容因此未进入网站配置的正常保存流程。



(a) 在后台字段中提交 XSS 测试载荷



(b) 代理返回 HTTP 403

图 3.10 真实业务中的 XSS 请求提交与阻断结果

阻断事件会同步进入运行控制台。图 3.11 中的最新安全事件记录了请求时间、客户端、POST /admin/set.php、处理结果以及 xss_script_open、xss_alert 等攻击证据，控制台同时汇总请求总量、攻击数量、阻断数量和变化趋势。由此，管理人员

不仅能够确认请求是否被阻断，还能进一步定位攻击发生的请求和触发判断的主要依据。

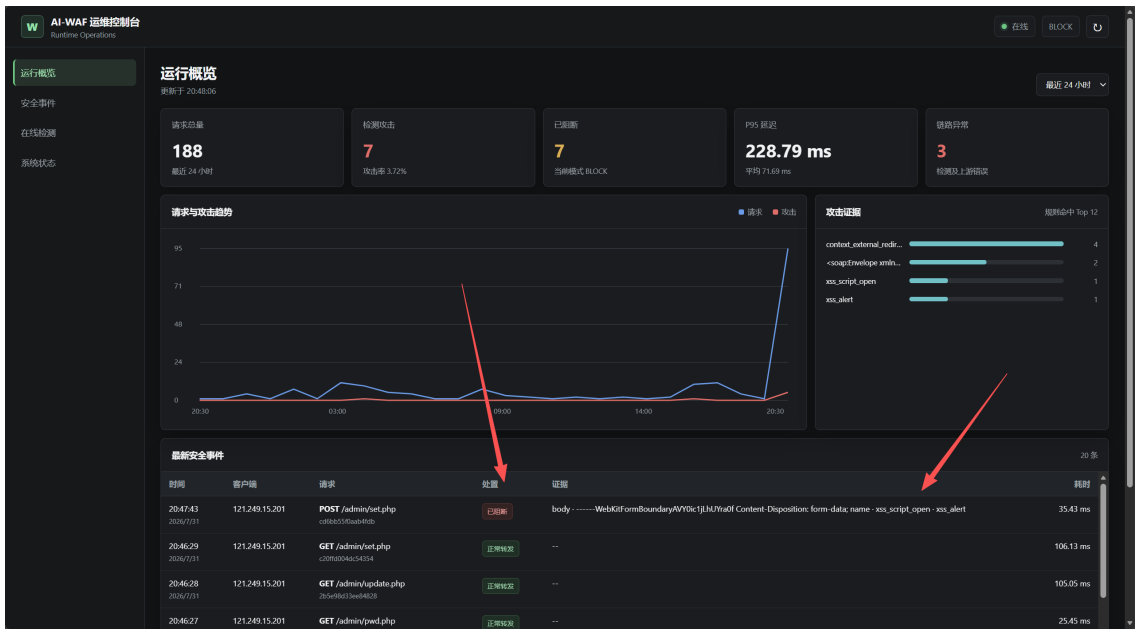


图 3.11 运行控制台记录真实业务中的 XSS 阻断事件

运行控制台还提供模式切换和状态检查，如图 3.12 所示。在 monitor 模式下，系统检测并记录攻击，但继续将请求转发至上游业务；在 block 模式下，系统保留监控记录，同时对攻击请求返回 403。页面还分别展示代理在线状态、上游业务、模型加载情况和故障策略。运行模式负责决定“检测到攻击后是否阻断”，故障策略负责决定“检测程序异常时是否继续转发”，两者相互独立。

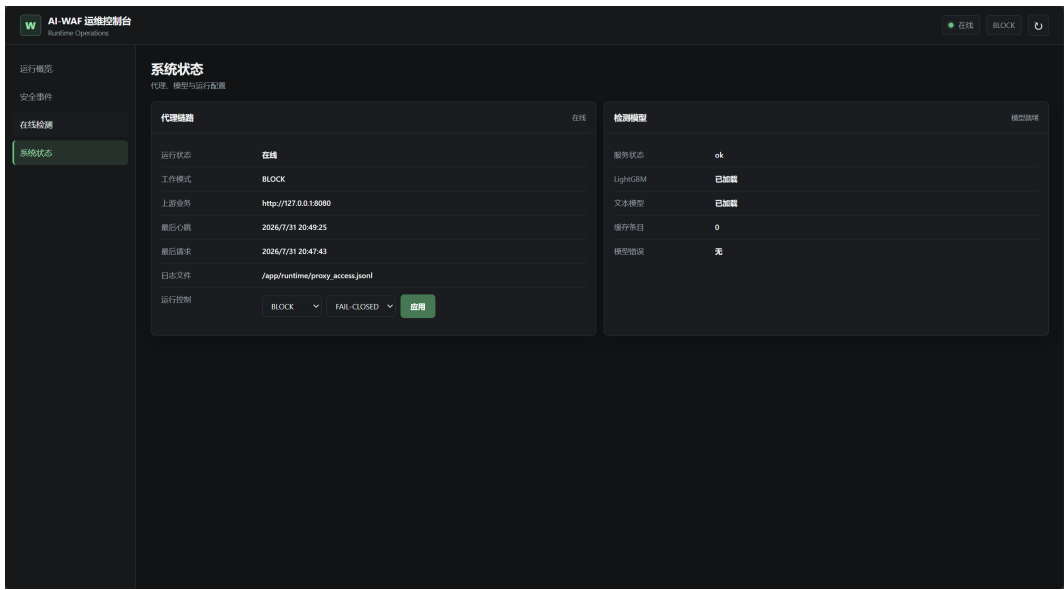


图 3.12 真实业务代理的运行状态与模式控制

需要说明的是，运行控制台显示的是代理链路的整体处理时间，不能直接视为模型的单次检测耗时。该时间覆盖请求接收、HTTP 解析、字段提取、递归归一化、规则与模型判断和事件记录；对于正常转发的请求，还包含上游连接和响应过程。服务器计算性能、网络状态、并发请求以及业务页面产生的多项静态资源请求也会影响控制台中的平均延迟和 P95 延迟。因此，图 3.11 中的事件耗时和控制台总体延迟用于反映真实服务器上的代理链路状态，模型自身性能仍以本节载荷级独立测试得到的 P99 1.8383 ms 为准。

本次测试形成了“真实请求接入—攻击字段提取—XSS 识别—请求级判定—HTTP 403 阻断—安全事件记录”的完整证据链，说明系统已经能够将离线验证的检测能力应用于真实 Web 业务，并根据部署需要在只监控和监控加阻断两种方式之间切换。

第四章 创新性说明

本作品围绕混淆攻击在内容还原、特征识别、数据覆盖和在线处置中的关键问题，从归一化与多视图检测、规则与双模型协同、多源混淆数据构建以及请求级防护四个方面进行设计，形成面向复杂混淆载荷的完整检测与防护链路。

4.1 归一化与多视图检测

混淆攻击常通过多层 URL 编码、Unicode 转义、HTML 实体替换、注释插入和字符拆分等方式改变载荷表面形式，使原有攻击结构被隐藏。本作品设计了有边界的递归归一化方法，对编码和结构变形进行逐层恢复，并在内容稳定或达到处理上限后停止，提升对多层混淆载荷的还原能力。

在归一化基础上，系统进一步构建原始视图与归一化视图。原始视图保留异常编码、控制字符、特殊转义等攻击痕迹，归一化视图呈现还原后的关键词、字符组合和攻击结构。两种视图分别参与规则和模型检测，使系统能够从载荷原始表现和还原结构两个角度提取风险信息。

正式测试集中包含 1,357 条混淆攻击，完整检测链路召回率达到 98.8946%，与原始攻击 98.8014% 的召回率保持在同一水平，体现了系统在复杂编码和结构变形条件下的稳定检测能力。

4.2 规则与双模型协同

本作品构建了由高置信规则、LightGBM 结构模型和字符 n-gram 模型组成的协同检测机制。高置信规则关注具有明确攻击含义的关键结构，LightGBM 模型利用编码次数、特殊符号、长度变化和危险结构等特征描述载荷整体形态，字符 n-gram 模型从局部字符组合中捕获拆分、替换和编码后仍然保留的攻击模式。

检测过程中，各模块按照不同侧重点共同参与风险判断。双模型分别输出风险概率，并依据验证实验确定融合权重，将整体结构信息与局部字符模式进行综合判断，同时结合规则提供的明确攻击证据，形成多层次的检测结果。

组件对比实验中，双模型融合的精确率、召回率和 F1 分别达到 99.5988%、99.3000% 和 99.4492%，F1 分别比仅使用 LightGBM 和仅使用字符模型提高 0.1520 和 0.3968 个百分点，体现了不同类型特征之间的互补作用。

4.3 多源混淆数据构建

本作品从数据来源和攻击变形两个维度扩展训练与评测数据。数据覆盖公开安全语料、隔离靶场采集、近期漏洞相关请求、项目生成数据、正常业务样本和混淆派生数据，使攻击样本与正常样本具有更丰富的来源和内容形式。

在攻击变形方面，项目共整理并实现 58 种混淆与变形方法，覆盖 URL 编码、Unicode 转义、HTML 实体、大小写扰动、空白填充、注释插入、字符拆分、路径变形、命令结构变化等多种方式，并针对不同攻击类型设计对应的变形方法。多种方法还可以继续组合，形成更复杂的多层混淆载荷，从而扩展同一攻击在不同表现形式下的样本覆盖。

数据集中同时加入 SQL 教程、程序代码、文件路径、富文本、安全讨论等容易包含敏感字符或攻击关键词的正常样本，使模型能够学习攻击结构与正常技术文本之间的差异。样本构建过程保留来源、攻击类型、变换方式、变换顺序和文件指纹等信息，为训练复现、独立评测和漏报分析提供统一的数据基础。当前组织后的数据规模达到 639,984 条。

4.4 请求级防护闭环

本作品将载荷检测进一步扩展到完整 Web 请求。系统分别提取路径、参数、请求体、请求头和 Cookie 等字段，对其中的多个载荷执行归一化、规则判断和模型检测，并保留各检测结果对应的字段来源。

各字段检测结果经过统一汇总后形成请求级风险结论，并进一步映射为 monitor 和 block 两类在线处置动作。monitor 模式记录风险字段、归一化内容和判定结果后继续转发请求，block 模式直接完成请求拦截。

系统同时记录原始载荷、归一化结果、风险字段、检测依据、请求结论和处置动作，使单载荷识别、多字段汇总和在线处置保持完整关联，形成从攻击检测到实际防护的请求级闭环。

上述四方面共同组成 AI-WAF 的创新体系：归一化与多视图提升混淆还原能力，规则与双模型强化多层次风险识别，多源与 58 种混淆方法扩大数据覆盖，请求级闭环进一步将检测能力落实到实际在线防护。

第五章 总结

本文针对 Web 攻击载荷通过多层编码、注释插入、字符拆分和结构变形逃逸检测的问题，设计并实现了 AI-WAF。系统以完整 HTTP 请求为对象，从路径、参数、请求体、请求头和 Cookie 中提取候选载荷，保留字段来源与原始证据，并通过有边界的递归归一化恢复隐藏结构。检测阶段由高置信规则处理明确样本，LightGBM 结构模型与字符 n-gram 文本模型分析边界样本，再结合字段上下文和多载荷结果形成请求级结论。多来源数据、可追溯混淆生成、正常难负例和按组划分的数据分区，为模型学习攻击语义而非记忆固定关键词提供了基础。

在 34,721 条独立测试记录上，完整检测链路的精确率、召回率和 F1 分别为 99.0476%、98.8515% 和 98.9495%，误报率为 0.0745%。系统对 1,357 条混淆攻击的召回率为 98.8946%，与原始攻击的 98.8014% 基本一致，说明编码嵌套、大小写扰动、注释插入和字符串拆分未造成明显性能下降。组件消融表明，递归归一化直接提高了规则路径对混淆攻击的覆盖，结构模型与字符模型在速度、召回和误报控制上具有互补性。统一数据与判定口径下，系统 F1 高于 ModSecurity + OWASP CRS、open-appsec 和 SafeLine 三个固定版本产品；载荷级平均延迟为 0.7933 ms，P99 为 1.8383 ms，满足在线处理目标。

完整请求与真实业务测试进一步验证了系统能够定位风险字段、区分正常技术文本与活动攻击，并将请求级结论转化为监控记录、正常转发或 HTTP 403 阻断。总体而言，本作品通过多视图还原、规则与双模型融合及请求上下文汇总，较好地回应了混淆鲁棒性、检测准确性和在线实时性三项挑战，可用于 Web 入口防护、现有 WAF 的补充分析以及授权环境中的混淆攻击鲁棒性测试。

参考文献

- [1] OWASP Foundation. Web Security Testing Guide[EB/OL]. [2026-08-01]. <https://owasp.org/www-project-web-security-testing-guide/>.
- [2] OWASP Foundation. Encoded Injection[EB/OL]. [2026-08-01]. https://owasp.org/www-project-web-security-testing-guide/latest/6-Appendix/D-Encoded_Injection.
- [3] OWASP Foundation. Double Encoding[EB/OL]. [2026-08-01]. https://owasp.org/www-community/Double_Encoding.
- [4] MITRE. CWE-89: Improper Neutralization of Special Elements used in an SQL Command[EB/OL]. [2026-08-01]. <https://cwe.mitre.org/data/definitions/89.html>.
- [5] MITRE. CWE-79: Improper Neutralization of Input During Web Page Generation[EB/OL]. [2026-08-01]. <https://cwe.mitre.org/data/definitions/79.html>.
- [6] MITRE. CWE-78: Improper Neutralization of Special Elements used in an OS Command[EB/OL]. [2026-08-01]. <https://cwe.mitre.org/data/definitions/78.html>.
- [7] HANDLEY M, PAXSON V, KREIBICH C. Network Intrusion Detection: Evasion, Traffic Normalization, and End-to-End Protocol Semantics[C]// Proceedings of the 10th USENIX Security Symposium. Washington, D.C.: USENIX Association, 2001.
- [8] OWASP Foundation. OWASP Core Rule Set[EB/OL]. [2026-08-01]. <https://owasp.org/www-project-modsecurity-core-rule-set/>.
- [9] BERNERS-LEE T, FIELDING R, MASINTER L. Uniform Resource Identifier (URI): Generic Syntax: RFC 3986[S/OL]. IETF, 2005. DOI: 10.17487/RFC3986. [2026-08-01]. <https://www.rfc-editor.org/info/rfc3986>.
- [10] OWASP Foundation. Testing for HTTP Parameter Pollution[EB/OL]. OWASP Web Security Testing Guide. [2026-08-01]. <https://owasp.org/>

[www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/04-Testing_for_HTTP_Parameter_Pollution](https://www.project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/04-Testing_for_HTTP_Parameter_Pollution).

- [11] National Institute of Standards and Technology. National Vulnerability Database[EB/OL]. [2026-08-01]. <https://www.nist.gov/itl/nvd>.
- [12] OWASP ModSecurity Project. ModSecurity Reference Manual (v3.x): Processing Phases[EB/OL]. [2026-08-01]. <https://github.com/owasp-modsecurity/ModSecurity/wiki/Reference-Manual-%28v3.x%29>.
- [13] OWASP Core Rule Set Project. Anomaly Scoring[EB/OL]. [2026-08-01]. https://coreruleset.org/docs/2-how-crs-works/2-1-anomaly_scoring/.
- [14] open-appsec. Contextual Machine Learning[EB/OL]. [2026-08-01]. <https://docs.openappsec.io/concepts/contextual-machine-learning>.
- [15] CHAITIN TECH. SafeLine: A Self-hosted Web Application Firewall[EB/OL]. [2026-08-01]. <https://github.com/chaitin/safeline>.
- [16] DOMINGUES R S, EBECKEN N F F. A new WAF architecture with machine learning for resource-efficient use[J]. Computers & Security, 2021, 106: 102290. DOI: 10.1016/j.cose.2021.102290.
- [17] DEMETRIO L, VALENZA A, COSTA G, et al. WAF-A-MoLE: Evading Web Application Firewalls through Adversarial Machine Learning[C]// Proceedings of the 35th Annual ACM Symposium on Applied Computing. New York: ACM, 2020: 1745–1752. DOI: 10.1145/3341105.3373962.
- [18] QU Z, LING X, WANG T, et al. AdvSQLi: Generating Adversarial SQL Injections against Real-World WAF-as-a-Service[J]. IEEE Transactions on Information Forensics and Security, 2024, 19: 2623–2638. DOI: 10.1109/TIFS.2024.3350911.
- [19] AKHAVANI S A, JABIYEV B, KALLUS N, et al. WAFFLED: Exploiting Parsing Discrepancies to Bypass Web Application Firewalls[EB/OL]. arXiv:2503.10846, 2025. [2026-08-01]. <https://arxiv.org/abs/2503.10846>.

- [20] ALAHMADI B A, AXON L, MARTINOVIC I. 99% False Positives: A Qualitative Study of SOC Analysts' Perspectives on Security Alarms[C]// Proceedings of the 31st USENIX Security Symposium. Berkeley: USENIX Association, 2022: 2783–2800. [2026-08-01]. <https://www.usenix.org/conference/usenixsecurity22/presentation/alahmadi>.
- [21] KE G, MENG Q, FINLEY T, et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree[C]// Advances in Neural Information Processing Systems 30. Red Hook: Curran Associates, 2017: 3146–3154.
- [22] SALTON G, BUCKLEY C. Term-weighting approaches in automatic text retrieval[J]. Information Processing & Management, 1988, 24(5): 513–523.
- [23] PAN Y, SUN F, TENG Z, et al. Detecting web attacks with end-to-end deep learning[J]. Journal of Internet Services and Applications, 2019, 10: 16. DOI: 10.1186/s13174-019-0115-x.
- [24] SWISSKYREPO. PayloadsAllTheThings[EB/OL]. [2026-08-01]. <https://github.com/swisskyrepo/PayloadsAllTheThings>.
- [25] FUZZDB PROJECT. fuzzdb: Dictionary of attack patterns and primitives for black-box application fault injection and resource discovery[EB/OL]. [2026-08-01]. <https://github.com/fuzzdb-project/fuzzdb>.